Micron Open Day at IIT Bombay — Hackathon

24 October, 2026

Problem Statement:

1. You are provided with a module with 14 pads - each pad having 70umx70um size and 140um pitch. You are tasked to build a set of test structures for on-chip measurement of MOSFETs using these pads to connect to its terminals. You must measure ID-VG and ID-VD data for a given MOSFET with 4 terminals (G,D,S,B). Fit as many MOSFETs as possible in this 14-pad module with the constraint of minimizing the effect of interconnect resistance between the pad and the device terminals.

2. You are provided with three identical MOSFETs with four terminals each (G,D,S,B) placed next to each other as shown in the device schematic. Propose a measurement methodology that can accurately extract the LDD resistance from this structure. Assume $R_{NiSi} << R_{LDD}$.

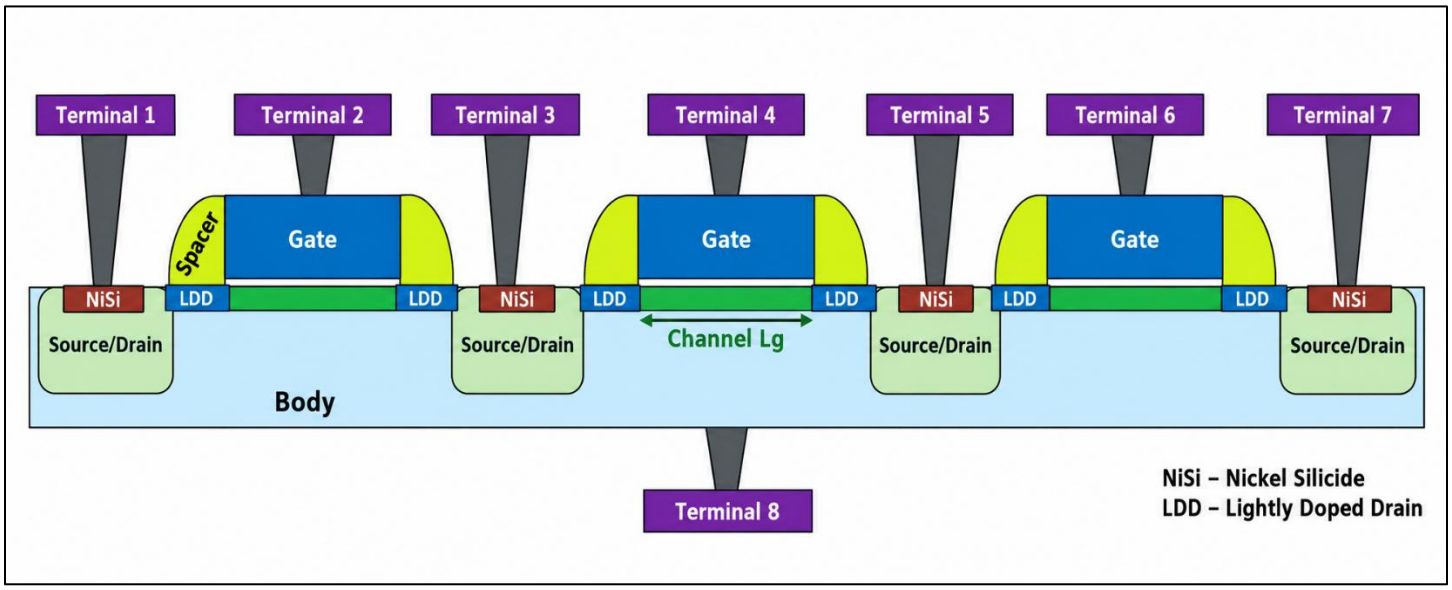


Figure 1. Schematic cross-section of three adjacent MOSFETs showing the source/drain regions, nickel silicide (NiSi), lightly doped drain (LDD) extensions, gate electrodes, channel lengths, and terminal assignments used in the LDD-resistance extraction problem.

Submitted by:

Team name: Silicon Sleuths

1. Kusumuru Suresh Kumar (✉ kusumurusureshkumar21@gmail.com) (☎ 9182592794)
   26M2029, M. Tech 1st Year, Systems and Control Engineering
   IIT Bombay

2. Karan Ahirwar (✉ karanahirwar32128@gmail.com) (☎ 9981789428)
   26M2022, M. Tech 1st Year, Systems and Control Engineering
   IIT Bombay

Find it on the GitHub at https://github.com/suresh-vlsi/Micron-Open-Day-at-IIT-Bombay-Hackathon

Problem 1 — Solution:

The maximum is 5 MOSFETs, using all 14 pads.

Key assumptions

- All DUTs are the same type (say NMOS), so they can share one substrate/body.
- Gate and bulk draw essentially zero DC current, so there is no IR drop on those lines and no sense line is needed for them.
- Drain and source carry the full ID, so interconnect resistance matters there. Each needs a force pad (carries current) and a sense pad (high-impedance voltage probe, ~0 current). This is a Kelvin connection.
- The sense and force lines for a terminal are joined right at the device terminal, not at the pad, so the measured voltage excludes the interconnect resistance.
- Only one DUT is measured at a time. The others have floating drains, so they carry no current. With VDS = 0 and a floating drain, an idle device is effectively inactive, apart from negligible junction leakage.

Why 5 is the maximum? — Justification

1. Shared pads: G, B, S-force and S-sense, which is 4 pads.
2. Each DUT needs its own D-force and D-sense, which is 2 pads.

$4 + 2n \leq 14$ gives $n = 5$, using exactly 14 pads.

Pad allocation:

Pads	Connection
1	Common Gate (G)
2	Common Bulk (B)
3	Common Source Force (S_F)
4	Common Source Sense (S_S)
5, 6	M1 Drain Force / Sense
7, 8	M2 Drain Force / Sense
9, 10	M3 Drain Force / Sense
11, 12	M4 Drain Force / Sense
13, 14	M5 Drain Force / Sense

Measurement procedure

- ID–VG: Use an SMU on the D_F pad of the selected device to hold VD fixed (for example, 50 mV for linear or VDD for saturation). Use an SMU on the G pad to sweep VG. Use an SMU on S_F to hold the source at 0 V and measure ID. Ground B or set it to the desired VB. Read the true VD and VS through the sense pads (the SMU's remote-sense inputs adjust the forced voltages automatically).
- ID–VD: Fix VG at several values and sweep VD on D_F. Measure the current at S_F or D_F.
- Repeat for M2 through M5 by moving the drain SMU to that device's D pads. The other drains stay floating (high impedance).

Layout notes

- The pad pitch is 140 µm with 70 µm pads, leaving 70 µm gaps. A single row of 14 pads fits the 14 × 140 µm span, so the shared G, B and S pads can sit at one end, with the drain pairs placed adjacent to each DUT.
- Route force lines in wide metal (low resistance, electromigration-safe). Route sense lines as minimum-width traces that tap the terminal directly, close to the diffusion contact.
- Place the 5 DUTs close together and as near the pads as possible to keep the shared source/bulk routing short.
- Use identical routing for each DUT's drain pair so the measurements are comparable.

Schematic Macro code for freeCAD tool:

```
# -*- coding: utf-8 -*-
"""
MOSFET_Between_Pads_Dimensioned.FCMacro

Same layout as MOSFET_Between_Pads.FCMacro (14 pads, 70 x 70 um, 140 um pitch,
5 NMOS DUTs placed in the 70 um gaps between each D_force / D_sense pad pair,
Kelvin drain + source, shared G/B/S buses) PLUS:

1. A dimensioned top view (pad size, pitch, gap, device positions, total
footprint) and an 8:1 DETAIL A of the M1 gap (routing widths, spacings,
device structure). Dimension values are TRUE um, not drawing units.
2. A design-rule table with calculated margins (FreeCAD drawing + Spreadsheet
+ Report view).
3. A concept-vs-foundry comparison table and an on-drawing banner.

============================== READ THIS FIRST ==============================
This is a CONCEPTUAL FreeCAD VISUALIZATION, not a foundry-rule-compliant
physical layout.

* The rule values in RULES are generic PLACEHOLDERS, not from any PDK.
"OK" against them does NOT mean DRC/LVS clean at a foundry.
* No wells/implants/STI/guard rings, no passivation opening, no GDS layer
numbers, no manufacturing grid, single oversized contacts/vias instead of
exact-size arrays. The table marks these as NON-COMPLIANT on purpose.
* The real layout must be drawn/verified in the foundry PDK flow
(KLayout / Virtuoso etc.: DRC, LVS, parasitic extraction, EM/IR).
================================================================================

Units: all coordinates are um; converted to FreeCAD mm (1 um = 0.001 mm).
Run: Macro > Macros... > select file > Execute. Zoom in (View > Fit all) -
text is um-scale.
"""

import math
import os
import FreeCAD as App
import Part
from FreeCAD import Vector

UM = 0.001

# ------------------------------ given (problem) ----------------------------
PAD_GIVEN = 70.0
PITCH_GIVEN = 140.0
PAD = PAD_GIVEN
PITCH = PITCH_GIVEN
N_PADS = 14
N_DUT = 5

# ------------------------------ design parameters (um) ---------------------
W_ACT = 20.0 # MOSFET width W
L_GATE = 2.0 # MOSFET length L
W_FORCE = 6.0 # force trace width
W_SENSE = 2.0 # sense trace width
W_STUB = 4.0 # gate / bulk / source-force stub width
VIA_S = 4.0 # drawn via size (concept: single big via)
LAND_S = 6.0 # metal landing size around a via
LABEL_SIZE = 7.0 # pad label text height (um)

Y_BUS = {"G": -60.0, "B": -80.0, "SF": -100.0, "SS": -120.0}
BUS_W = {"G": W_FORCE, "B": W_FORCE, "SF": W_FORCE, "SS": W_SENSE}

Y_SRC = (-14.0, 6.0)
Y_GATE = (6.0, 6.0 + L_GATE)
Y_DRN = (Y_GATE[1], Y_GATE[1] + 20.0)
Y_SRC_C = -4.0
Y_GATE_C = Y_GATE[0] + L_GATE / 2
Y_DRN_C = Y_DRN[0] + 10.0

X_STUB = {"B": -28.0, "G": -18.0, "SF": -4.0, "SS": 4.0}
W_NET = {"B": W_STUB, "G": W_STUB, "SF": W_STUB, "SS": W_SENSE}

# -------- PLACEHOLDER rule set + test conditions (NOT from any foundry) ----
RULES = dict(
M1_W=0.5, M1_S=0.5, M2_W=0.6, M2_S=0.6,
POLY_W=0.18, ENDCAP=0.25, DIFF_W=0.3, DIFF_S=0.4, POLY_CON_S=0.15,
CON_SIZE=0.25, VIA_SIZE=0.5, MET_ENCL_VIA=0.2, MAX_WIDE_METAL=12.0,
```

```
PAD_MIN_SIZE=50.0, PAD_MIN_PITCH=100.0, PAD_MIN_SPACE=25.0,
ACTIVE_TO_PAD=10.0,
J_M1=1.0, J_M2=2.0, # mA per um of width (assumed EM limits)
RS_M1=0.09, RS_M2=0.08, # ohm/square (assumed)
)
I_TEST_MA = 5.0 # assumed max DUT current during the test

# -------------------------------- appearance --------------------------------
DIM_Z = 3.0
DIM_TEXT = 12.0 # text height of the overall-view dimensions (um)
DET_TEXT = 9.0 # text height inside DETAIL A (um)
TABLE_TEXT = 10.0 # table text height (um)
EXPORT_CSV = False # True: write every rectangle to ~/mosfet_test_module_rects.csv

Z = {
"SUB": (-1.0, 0.0), "DIFF": (0.0, 0.2), "POLY": (0.2, 0.4),
"M1": (1.0, 1.5), "VIA": (1.5, 2.0), "M2": (2.0, 2.5),
"CON": (0.2, 2.0),
}
COLORS = {
"SUB": (0.80, 0.80, 0.80), "DIFF": (0.20, 0.70, 0.20),
"POLY": (0.85, 0.20, 0.20), "M1": (0.20, 0.40, 0.90),
"VIA": (0.10, 0.10, 0.10), "M2": (0.95, 0.75, 0.10),
"PADS": (1.00, 0.85, 0.20), "CON": (0.10, 0.10, 0.10),
"DIM": (0.85, 0.05, 0.05), "TABLE": (0.0, 0.0, 0.0),
}

# -------------------------------- document --------------------------------
doc = App.newDocument("MOSFET_Between_Pads_Dimensioned")
groups = {n: doc.addObject("App::DocumentObjectGroup", "L_" + n) for n in COLORS}
rects_all = [] # every solid rectangle (um) - used for DRC, bbox, detail, export


# ------------------------------ geometry helpers ------------------------------
def _solid(layer, label, x0, y0, x1, y1, z0, z1):
obj = doc.addObject("Part::Feature", label)
obj.Shape = Part.makeBox((x1 - x0) * UM, (y1 - y0) * UM, (z1 - z0) * UM,
Vector(x0 * UM, y0 * UM, z0 * UM))
obj.Label = label
groups[layer].addObject(obj)
if App.GuiUp:
obj.ViewObject.ShapeColor = COLORS[layer]
return obj


def box(layer, label, x0, y0, x1, y1, net=None, zkey=None, z=None, kind="geom"):
xa, xb = sorted((x0, x1))
ya, yb = sorted((y0, y1))
z0, z1 = z if z else Z[zkey or layer]
obj = _solid(layer, label, xa, ya, xb, yb, z0, z1)
rects_all.append(dict(layer=layer, label=label, net=net, kind=kind,
x0=xa, y0=ya, x1=xb, y1=yb, z0=z0, z1=z1))
return obj


def vline(layer, label, x, y0, y1, w, net):
return box(layer, label, x - w / 2, y0, x + w / 2, y1, net)


def hline(layer, label, y, x0, x1, w, net):
return box(layer, label, x0, y - w / 2, x1, y + w / 2, net)


def sq(layer, label, x, y, s, net=None, z=None):
return box(layer, label, x - s / 2, y - s / 2, x + s / 2, y + s / 2, net, z=z)


def contact(label, x, y, s, start_layer):
sq("CON", label, x, y, s, z=(Z[start_layer][1], Z["CON"][1]))


def bus_via(net, x, y, tag):
sq("M1", "M1LAND_" + tag, x, y, LAND_S, net)
sq("VIA", "VIA_" + tag, x, y, VIA_S)
sq("M2", "M2LAND_" + tag, x, y, LAND_S, net)


def dl(r):
return "M2" if r["layer"] in ("M2", "PADS") else r["layer"]
```

```
def rect_dist(a, b):
dx = max(a["x0"] - b["x1"], b["x0"] - a["x1"], 0.0)
dy = max(a["y0"] - b["y1"], b["y0"] - a["y1"], 0.0)
return math.hypot(dx, dy)


def by_label(label):
for r in rects_all:
if r["label"] == label:
return r
raise KeyError(label)


# ---------------------------- annotation helpers ----------------------------
def add_label(text, x_um, y_um, size_um, just="Center", color=(0.0, 0.0, 0.0)):
"""Draft text of height size_um (um) anchored at (x, y) in um."""
import Draft
make = getattr(Draft, "make_text", None) or getattr(Draft, "makeText")
t = make([text], Vector(x_um * UM, y_um * UM, DIM_Z * UM))
if App.GuiUp:
vo = t.ViewObject
vo.FontSize = size_um * UM
for prop, val in (("Justification", just), ("TextColor", color)):
try:
setattr(vo, prop, val)
except Exception:
pass
return t


def seg(x0, y0, x1, y1):
return Part.makeLine(Vector(x0 * UM, y0 * UM, DIM_Z * UM),
Vector(x1 * UM, y1 * UM, DIM_Z * UM))


def rect_edges(x0, y0, x1, y1):
return [seg(x0, y0, x1, y0), seg(x1, y0, x1, y1),
seg(x1, y1, x0, y1), seg(x0, y1, x0, y0)]


def edges_obj(label, edges, group="DIM", color=None, width=1.0):
obj = doc.addObject("Part::Feature", label)
obj.Shape = Part.makeCompound(edges)
obj.Label = label
groups[group].addObject(obj)
if App.GuiUp:
obj.ViewObject.LineColor = color or COLORS[group]
obj.ViewObject.LineWidth = width
return obj


def _pick(v, i):
return v[i] if isinstance(v, (list, tuple)) else v


def hchain(label, xs, y, labels, size, ext_from=None, ext_over=3.0, alt=False):
"""Horizontal dimension chain. Positions in drawing um, labels are text."""
t = size * 0.35
edges = [seg(xs[0], y, xs[-1], y)]
for i, x in enumerate(xs):
edges.append(seg(x - t, y - t, x + t, y + t))
ef = _pick(ext_from, i)
if ef is not None:
edges.append(seg(x, ef, x, y + ext_over if ef < y else y - ext_over))
edges_obj(label, edges)
for i, txt in enumerate(labels):
ty = y - size * 1.6 if (alt and i % 2 == 1) else y + size * 0.5
add_label(txt, (xs[i] + xs[i + 1]) / 2, ty, size, color=(0.7, 0, 0))


def vchain(label, ys, x, labels, size, ext_from=None, ext_over=3.0, side=1):
"""Vertical dimension chain; text on the +x side (side=1) or -x side (-1)."""
t = size * 0.35
edges = [seg(x, ys[0], x, ys[-1])]
for i, y in enumerate(ys):
edges.append(seg(x - t, y - t, x + t, y + t))
ef = _pick(ext_from, i)
```

```
if ef is not None:
edges.append(seg(ef, y, x + ext_over if ef < x else x - ext_over, y))
edges_obj(label, edges)
for i, txt in enumerate(labels):
add_label(txt, x + side * size * 0.7, (ys[i] + ys[i + 1]) / 2 - size / 2,
size, just="Left" if side > 0 else "Right", color=(0.7, 0, 0))


def draw_table(title, rows, x0, y_top, size):
"""Draw a grid table with one text per cell. Returns the bottom y."""
pad = size * 0.5
ncol = len(rows[0])
widths = [max(len(str(r[c])) for r in rows) * size * 0.62 + 2 * pad
for c in range(ncol)]
rh = size * 1.9
width, height = sum(widths), rh * len(rows)
edges = [seg(x0, y_top - i * rh, x0 + width, y_top - i * rh)
for i in range(len(rows) + 1)]
xc = x0
for c in range(ncol + 1):
edges.append(seg(xc, y_top, xc, y_top - height))
if c < ncol:
xc += widths[c]
edges_obj("TABLE_" + title.split()[0], edges, "TABLE", width=1.0)
add_label(title, x0, y_top + size * 0.8, size * 1.25, just="Left")
for r, row in enumerate(rows):
xc = x0
for c, cell in enumerate(row):
add_label(str(cell), xc + pad, y_top - (r + 1) * rh + (rh - size) / 2,
size, just="Left")
xc += widths[c]
return y_top - height


def to_sheet(name, rows):
try:
sh = doc.addObject("Spreadsheet::Sheet", name)
for r, row in enumerate(rows):
for c, cell in enumerate(row):
sh.set("%s%d" % (chr(65 + c), r + 1), "'" + str(cell))
return sh
except Exception as e:
App.Console.PrintWarning("Spreadsheet %s skipped: %s\n" % (name, e))


def print_table(title, rows):
widths = [max(len(str(r[c])) for r in rows) for c in range(len(rows[0]))]
App.Console.PrintMessage("\n== %s ==\n" % title)
for i, row in enumerate(rows):
App.Console.PrintMessage(
" | ".join(str(c).ljust(widths[k]) for k, c in enumerate(row)) + "\n")
if i == 0:
App.Console.PrintMessage("-+-".join("-" * w for w in widths) + "\n")


def fmt(v):
return "%g" % round(v, 3)


# ============================== GEOMETRY ====================================
labels = ["G", "B", "SF", "SS"]
for k in range(1, N_DUT + 1):
labels += ["M%d_DF" % k, "M%d_DS" % k]
assert len(labels) == N_PADS
pad_x = [i * PITCH for i in range(N_PADS)]


def pad_net(lab):
return lab.split("_")[0] + "_D" if lab.startswith("M") else lab


box("SUB", "SUBSTRATE_B", pad_x[0] - 100, Y_BUS["SS"] - 40,
pad_x[-1] + 100, PAD / 2 + 45, zkey="SUB", kind="sub")

for i, lab in enumerate(labels):
box("PADS", "PAD%02d_%s" % (i + 1, lab), pad_x[i] - PAD / 2, -PAD / 2,
pad_x[i] + PAD / 2, PAD / 2, net=pad_net(lab), zkey="M2", kind="pad")

dev_cx = [pad_x[4 + 2 * k] + PITCH / 2 for k in range(N_DUT)]
```

```
for net, idx in (("G", 0), ("B", 1), ("SF", 2), ("SS", 3)):
xp, yb = pad_x[idx], Y_BUS[net]
w = BUS_W[net]
vline("M2", "DROP_" + net, xp, yb, 0.0, w, net)
bus_via(net, xp, yb, "PAD_" + net)
x_end = max(cx + X_STUB[net] for cx in dev_cx)
hline("M1", "BUS_" + net, yb, xp - 3.0, x_end + 3.0, w, net)

for k, cx in enumerate(dev_cx, start=1):
t = "M%d" % k
dnet = t + "_D"
x_left_pad_edge = cx - PITCH / 2 + PAD / 2
x_right_pad_edge = cx + PITCH / 2 - PAD / 2

box("DIFF", t + "_ACTIVE", cx - W_ACT / 2, Y_SRC[0], cx + W_ACT / 2, Y_DRN[1])
box("DIFF", t + "_BULK_TAP", cx - 31.0, Y_SRC[0], cx - 25.0, Y_SRC[1])
box("POLY", t + "_GATE_POLY", cx - 21.0, Y_GATE[0], cx + W_ACT / 2 + 2.0,
Y_GATE[1])

xg = cx + X_STUB["G"]
vline("M2", t + "_G_STUB", xg, Y_GATE_C, Y_BUS["G"], W_NET["G"], "G")
bus_via("G", xg, Y_BUS["G"], t + "_G")
contact(t + "_G_CON", xg, Y_GATE_C, 4.0, "POLY")

xb = cx + X_STUB["B"]
vline("M2", t + "_B_STUB", xb, Y_SRC_C, Y_BUS["B"], W_NET["B"], "B")
bus_via("B", xb, Y_BUS["B"], t + "_B")
contact(t + "_B_CON", xb, Y_SRC_C, 4.0, "DIFF")

for net, lab in (("SF", "S_FORCE"), ("SS", "S_SENSE")):
xs = cx + X_STUB[net]
vline("M2", "%s_%s_STUB" % (t, lab), xs, Y_SRC_C, Y_BUS[net],
W_NET[net], net)
bus_via(net, xs, Y_BUS[net], "%s_%s" % (t, lab))
contact("%s_%s_CON" % (t, lab), xs, Y_SRC_C, 4.0, "DIFF")

hline("M2", t + "_DF_TRACE", Y_DRN_C, x_left_pad_edge - 3.0, cx + W_FORCE / 2,
W_FORCE, dnet)
hline("M2", t + "_DS_TRACE", Y_DRN_C, cx - W_SENSE / 2, x_right_pad_edge + 3.0,
W_SENSE, dnet)
contact(t + "_D_CON", cx, Y_DRN_C, 6.0, "DIFF")

# ============================ derived measurements ============================
metal = [r for r in rects_all if r["layer"] != "SUB"]
bx0 = min(r["x0"] for r in metal)
bx1 = max(r["x1"] for r in metal)
by0 = min(r["y0"] for r in metal)
by1 = max(r["y1"] for r in metal)
BW, BH = bx1 - bx0, by1 - by0


def min_dim(layers, skip_pads=True):
return min(min(r["x1"] - r["x0"], r["y1"] - r["y0"]) for r in rects_all
if r["layer"] in layers and not (skip_pads and r["kind"] == "pad"))


def max_trace_w(layers):
return max(min(r["x1"] - r["x0"], r["y1"] - r["y0"]) for r in rects_all
if r["layer"] in layers and r["kind"] != "pad")


def min_spacing(layer, pad_pairs_only=False):
items = [r for r in rects_all if r["net"] is not None and dl(r) == layer]
best, ba, bb, nviol = 1e9, None, None, 0
lim = RULES["M1_S"] if layer == "M1" else RULES["M2_S"]
for i in range(len(items)):
for j in range(i + 1, len(items)):
a, b = items[i], items[j]
if a["net"] == b["net"]:
continue
if pad_pairs_only and ((a["kind"] == "pad") == (b["kind"] == "pad")):
continue
d = rect_dist(a, b)
if d < lim:
nviol += 1
App.Console.PrintWarning("SPACING/SHORT %s: %s vs %s = %.2f um\n"
% (layer, a["label"], b["label"], d))
if d < best:
```

```
best, ba, bb = d, a, b
return best, ba, bb, nviol


m1_sp = min_spacing("M1")
m2_sp = min_spacing("M2")
pad_clear = min_spacing("M2", pad_pairs_only=True)
n_geom_viol = m1_sp[3] + m2_sp[3]

g1 = by_label("M1_ACTIVE")
tap1 = by_label("M1_BULK_TAP")
poly1 = by_label("M1_GATE_POLY")
endcap = poly1["x1"] - g1["x1"]
tap_sp = rect_dist(tap1, g1)
poly_con = min(rect_dist(poly1, by_label(n)) for n in
("M1_S_FORCE_CON", "M1_S_SENSE_CON", "M1_D_CON"))
act_pad = min(rect_dist(g1, by_label("PAD05_M1_DF")),
rect_dist(g1, by_label("PAD06_M1_DS")))
band_bot = g1["y0"] - (-PAD / 2)
band_top = PAD / 2 - g1["y1"]
con_min = min(r["x1"] - r["x0"] for r in rects_all if r["layer"] == "CON")

# worst-case (last device) source-force path and a drain-force path
cx5 = dev_cx[-1]
sq_drop = abs(Y_BUS["SF"]) / W_FORCE
sq_bus = ((cx5 + X_STUB["SF"]) - pad_x[2]) / W_FORCE
sq_stub = (Y_SRC_C - Y_BUS["SF"]) / W_STUB
r_src = (sq_drop + sq_stub) * RULES["RS_M2"] + sq_bus * RULES["RS_M1"]
r_drn = ((PITCH / 2 - PAD / 2 + 3.0 + W_FORCE / 2) / W_FORCE) * RULES["RS_M2"]

# ============================== DETAIL A (8:1) ==============================
DET_S, DET_X0, DET_YTOP = 8.0, 0.0, -320.0
cx1 = dev_cx[0]
WIN = (cx1 - 45.0, cx1 + 45.0, -130.0, 45.0)


def DX(x):
return DET_X0 + (x - WIN[0]) * DET_S


def DY(y):
return DET_YTOP - (WIN[3] - y) * DET_S


for r in list(rects_all):
x0, x1 = max(r["x0"], WIN[0]), min(r["x1"], WIN[1])
y0, y1 = max(r["y0"], WIN[2]), min(r["y1"], WIN[3])
if x1 <= x0 or y1 <= y0:
continue
_solid(r["layer"], "DET_" + r["label"], DX(x0), DY(y0), DX(x1), DY(y1),
r["z0"], r["z1"])

edges_obj("DET_FRAME", rect_edges(DX(WIN[0]), DY(WIN[2]), DX(WIN[1]), DY(WIN[3])),
"DIM", color=(0.0, 0.5, 0.0), width=2.0)
add_label("DETAIL A - M1 gap, 8:1 drawing scale (dimension values are TRUE um)",
DX(WIN[0]), DY(WIN[3]) + 14, DET_TEXT * 1.3, just="Left",
color=(0.0, 0.4, 0.0))

# gap between pads
hchain("DET_DIM_GAP", [DX(cx1 - 35), DX(cx1 + 35)], DY(41),
["gap %s" % fmt(PITCH - PAD)], DET_TEXT, ext_from=DY(PAD / 2))
# stub widths + spacings across the gap (row y = -47, crossing all four stubs)
xs, labs, prev = [DX(cx1 - PAD / 2)], [], -PAD / 2
for net in ("B", "G", "SF", "SS"):
left, right = X_STUB[net] - W_NET[net] / 2, X_STUB[net] + W_NET[net] / 2
xs += [DX(cx1 + left), DX(cx1 + right)]
labs += [fmt(left - prev), "%s %s" % (net, fmt(W_NET[net]))]
prev = right
xs.append(DX(cx1 + PAD / 2))
labs.append(fmt(PAD / 2 - prev))
hchain("DET_DIM_STUBS", xs, DY(-47), labs, DET_TEXT, alt=True)
# device width W
hchain("DET_DIM_W", [DX(cx1 - W_ACT / 2), DX(cx1 + W_ACT / 2)], DY(31),
["W = %s" % fmt(W_ACT)], DET_TEXT, ext_from=DY(Y_DRN[1]))
# device structure, vertical chain on the right
ys = [-PAD / 2, Y_SRC[0], Y_SRC[1], Y_GATE[1], Y_DRN[1], PAD / 2]
vl = [fmt(ys[1] - ys[0]), "S diff %s" % fmt(ys[2] - ys[1]),
"L = %s" % fmt(ys[3] - ys[2]), "D diff %s" % fmt(ys[4] - ys[3]),
fmt(ys[5] - ys[4])]
```

```
vchain("DET_DIM_DEV_Y", [DY(y) for y in ys], DX(cx1 + 40), vl, DET_TEXT,
ext_from=DX(cx1 + W_ACT / 2 + 2.0))
# M1 bus widths + spacings, vertical chain on the left (below the pad row)
yb_list, vlab, prev = [-PAD / 2], [], -PAD / 2
for net in ("G", "B", "SF", "SS"):
top, bot = Y_BUS[net] + BUS_W[net] / 2, Y_BUS[net] - BUS_W[net] / 2
yb_list += [top, bot]
vlab += [fmt(prev - top), "%s %s" % (net, fmt(BUS_W[net]))]
prev = bot
vchain("DET_DIM_BUS_Y", [DY(y) for y in yb_list], DX(cx1 - 40), vlab, DET_TEXT,
side=-1)
# drain force / sense trace widths
vchain("DET_DIM_DF", [DY(Y_DRN_C - W_FORCE / 2), DY(Y_DRN_C + W_FORCE / 2)],
DX(cx1 - 28), ["D_F %s" % fmt(W_FORCE)], DET_TEXT, side=-1)
vchain("DET_DIM_DS", [DY(Y_DRN_C - W_SENSE / 2), DY(Y_DRN_C + W_SENSE / 2)],
DX(cx1 + 22), ["D_S %s" % fmt(W_SENSE)], DET_TEXT, side=1)

# ====================== OVERALL DIMENSIONED TOP VIEW ======================
edges_obj("FOOTPRINT_OUTLINE", rect_edges(bx0, by0, bx1, by1), "DIM",
color=(0.7, 0.0, 0.7), width=1.5)
edges_obj("DETAIL_A_WINDOW", rect_edges(WIN[0], WIN[2], WIN[1], WIN[3]), "DIM",
color=(0.0, 0.5, 0.0), width=2.0)
add_label("A", WIN[1] + 8.0, WIN[2] + 5.0, DIM_TEXT * 1.4, just="Left",
color=(0.0, 0.4, 0.0))

p5, p6 = pad_x[4], pad_x[5]
hchain("DIM_PAD_W", [p5 - PAD / 2, p5 + PAD / 2], 100.0, ["pad %s" % fmt(PAD)],
DIM_TEXT, ext_from=PAD / 2)
hchain("DIM_PITCH", [p5, p6], 125.0, ["pitch %s" % fmt(PITCH)], DIM_TEXT,
ext_from=PAD / 2)
hchain("DIM_GAP", [p5 + PAD / 2, p6 - PAD / 2], 150.0,
["gap %s (M1 here)" % fmt(PITCH - PAD)], DIM_TEXT, ext_from=PAD / 2)
vchain("DIM_PAD_H", [-PAD / 2, PAD / 2], -60.0, ["pad %s" % fmt(PAD)], DIM_TEXT,
ext_from=-PAD / 2, side=-1)
dev_labels = ["M1 centre x=%s" % fmt(dev_cx[0])] + \
["%s" % fmt(dev_cx[i + 1] - dev_cx[i]) for i in range(N_DUT - 1)]
hchain("DIM_DEVICE_X", [0.0] + dev_cx, -150.0, dev_labels, DIM_TEXT,
ext_from=[-PAD / 2] + [Y_SRC[0]] * N_DUT)
hchain("DIM_FOOTPRINT_W", [bx0, bx1], -195.0,
["Total footprint W = %s um" % fmt(BW)], DIM_TEXT * 1.2,
ext_from=[-PAD / 2, -PAD / 2])
vchain("DIM_FOOTPRINT_H", [by0, by1], bx1 + 45.0, ["H = %s um" % fmt(BH)],
DIM_TEXT * 1.2, ext_from=[bx1, bx1], side=1)

# ================================ RULE TABLE ================================
rule_rows = [["Rule / quantity", "Layer", "Drawn", "Requirement", "Margin",
"Status", "Basis"]]


def _f(v, u):
return ("%.2f" % v) + (" " + u if u else "")


def rule_min(name, layer, drawn, req, basis, unit="um"):
m = drawn - req
rule_rows.append([name, layer, _f(drawn, unit), ">= " + _f(req, unit),
"%+.2f %s" % (m, unit), "OK" if m >= -1e-9 else "FAIL", basis])


def rule_max(name, layer, drawn, req, basis, unit="um"):
m = req - drawn
rule_rows.append([name, layer, _f(drawn, unit), "<= " + _f(req, unit),
"%+.2f %s" % (m, unit), "OK" if m >= -1e-9 else "FAIL", basis])


def rule_exact(name, layer, drawn, req, basis, unit="um"):
ok = abs(drawn - req) < 1e-9
rule_rows.append([name, layer, _f(drawn, unit), "= " + _f(req, unit),
"%+.2f %s" % (drawn - req, unit),
"OK" if ok else "NON-COMPLIANT", basis])


def info(name, layer, value, note):
rule_rows.append([name, layer, value, "-", "-", "INFO", note])


A, G = "ASSUMED rule", "GIVEN"
rule_exact("Pad size", "M2 pad", PAD, PAD_GIVEN, G)
rule_min("Pad size vs probe minimum", "M2 pad", PAD, RULES["PAD_MIN_SIZE"], A)
```

```
rule_exact("Pad pitch", "M2 pad", PITCH, PITCH_GIVEN, G)
rule_min("Pad pitch vs probe minimum", "M2 pad", PITCH, RULES["PAD_MIN_PITCH"], A)
rule_min("Pad-to-pad spacing (gap)", "M2 pad", PITCH - PAD, RULES["PAD_MIN_SPACE"], A)
rule_min("M1 minimum trace width", "M1", min_dim(["M1"]), RULES["M1_W"], A)
rule_min("M1 spacing (different nets)", "M1", m1_sp[0], RULES["M1_S"], A)
rule_min("M2 minimum trace width", "M2", min_dim(["M2"]), RULES["M2_W"], A)
rule_min("M2 spacing (different nets)", "M2", m2_sp[0], RULES["M2_S"], A)
rule_min("Stub/trace to pad-edge clearance", "M2", pad_clear[0], RULES["M2_S"], A)
rule_max("Widest non-pad metal (slotting)", "M1/M2", max_trace_w(["M1", "M2"]),
RULES["MAX_WIDE_METAL"], A)
rule_min("M1 enclosure of via", "M1", (LAND_S - VIA_S) / 2, RULES["MET_ENCL_VIA"], A)
rule_min("M2 enclosure of via", "M2", (LAND_S - VIA_S) / 2, RULES["MET_ENCL_VIA"], A)
rule_exact("Via size (single big via)", "VIA", VIA_S, RULES["VIA_SIZE"],
A + "; needs via array")
rule_exact("Contact size (single big contact)", "CON", con_min, RULES["CON_SIZE"],
A + "; needs contact array")
rule_min("Gate length L", "POLY", min_dim(["POLY"]), RULES["POLY_W"], A)
rule_min("Gate end-cap past active", "POLY", endcap, RULES["ENDCAP"], A)
rule_min("Diffusion minimum width", "DIFF", min_dim(["DIFF"]), RULES["DIFF_W"], A)
rule_min("Body-tap to active spacing", "DIFF", tap_sp, RULES["DIFF_S"], A)
rule_min("Poly to contact spacing", "POLY/CON", poly_con, RULES["POLY_CON_S"], A)
rule_min("Active to pad-edge (probe keep-out)", "DIFF/M2", act_pad,
RULES["ACTIVE_TO_PAD"], A)
rule_min("D-force trace current capacity", "M2", W_FORCE * RULES["J_M2"],
I_TEST_MA, "CALC, assumed J", unit="mA")
rule_min("S-force stub current capacity", "M2", W_STUB * RULES["J_M2"],
I_TEST_MA, "CALC, assumed J", unit="mA")
rule_min("S-force bus current capacity", "M1", BUS_W["SF"] * RULES["J_M1"],
I_TEST_MA, "CALC, assumed J", unit="mA")
info("Device keep-in margin to pad band (bottom / top)", "DIFF",
"%s / %s um" % (fmt(band_bot), fmt(band_top)), "CALC")
info("Stub span inside gap (free: left / right)", "M2",
"%s / %s um" % (fmt(X_STUB["B"] - W_NET["B"] / 2 + PAD / 2),
fmt(PAD / 2 - (X_STUB["SS"] + W_NET["SS"] / 2))), "CALC")
info("S-force path R, worst DUT (M5), est.", "M1+M2",
"%.1f ohm = %.0f mV @ %.0f mA" % (r_src, r_src * I_TEST_MA, I_TEST_MA),
"removed by Kelvin sense")
info("D-force path R, pad to drain, est.", "M2",
"%.2f ohm = %.1f mV @ %.0f mA" % (r_drn, r_drn * I_TEST_MA, I_TEST_MA),
"removed by Kelvin sense")
info("Pad-row extent", "M2 pad", "%s um" % fmt(pad_x[-1] - pad_x[0] + PAD), "CALC")
info("Total footprint (all layout geometry)", "all",
"%s x %s um = %.4f mm2" % (fmt(BW), fmt(BH), BW * BH * 1e-6), "CALC")
info("Assumed test current", "-", "%.1f mA" % I_TEST_MA, "ASSUMED")

n_ok = sum(1 for r in rule_rows[1:] if r[5] == "OK")
n_nc = sum(1 for r in rule_rows[1:] if r[5] == "NON-COMPLIANT")
n_fail = sum(1 for r in rule_rows[1:] if r[5] == "FAIL")

concept_rows = [
["Aspect", "This FreeCAD macro (CONCEPTUAL)", "Foundry-compliant physical layout"],
["Purpose", "Topology + geometry visualization", "Mask data (GDSII/OASIS) to fabricate"],
["Layers", "Generic SUB/DIFF/POLY/M1/VIA/M2 solids", "PDK layer/datatype set, wells, implants, STI, pad
opening"],
["Rules", "Placeholder values + simple spacing check", "Full foundry DRC deck, clean (incl. density, antenna)"],
["Contacts/vias", "Single 4-6 um blocks", "Exact-size contact/via arrays on the mfg grid"],
["Isolation", "None (one substrate block)", "Wells/deep-NWELL as needed, guard rings, latch-up taps"],
["Pads", "70 um squares on M2", "Pad cell: top-metal stack, passivation window, ESD/probe rules"],
["Grid", "Free-form um values", "Manufacturing grid snapping, angle rules"],
["Devices", "Rectangles with W and L", "PDK device cells with models, matching, parasitics"],
["Sign-off", "Visual review + this table", "DRC + LVS + PEX + EM/IR in the PDK flow"],
]

# ================================ TABLES / BANNER ============================
bottom = draw_table("DESIGN-RULE CONSTRAINTS AND MARGINS (PLACEHOLDER RULES)",
rule_rows, 800.0, DET_YTOP, TABLE_TEXT)
draw_table("CONCEPTUAL FREECAD MODEL vs FOUNDRY-COMPLIANT LAYOUT",
concept_rows, 800.0, bottom - 90.0, TABLE_TEXT)

add_label("CONCEPTUAL FreeCAD VISUALIZATION - NOT a foundry-compliant physical layout",
bx0, 235.0, 16.0, just="Left", color=(0.8, 0.0, 0.0))
add_label("Rule values are PLACEHOLDERS: passing them does NOT mean DRC/LVS clean. "
"Final layout must be built and signed off in the foundry PDK flow.",
bx0, 212.0, 10.0, just="Left", color=(0.6, 0.0, 0.0))

for i, lab in enumerate(labels):
add_label("P%d %s" % (i + 1, lab), pad_x[i], PAD / 2 + 8.0, LABEL_SIZE)
for k, cx in enumerate(dev_cx, start=1):
add_label("M%d" % k, cx, PAD / 2 + 8.0 + 2.5 * LABEL_SIZE, LABEL_SIZE * 1.4)
```

```
to_sheet("DRC_Margins", rule_rows)
to_sheet("Concept_vs_Foundry", concept_rows)

if EXPORT_CSV:
try:
path = os.path.join(os.path.expanduser("~"), "mosfet_test_module_rects.csv")
with open(path, "w") as fh:
fh.write("layer,label,net,x0_um,y0_um,x1_um,y1_um\n")
for r in rects_all:
fh.write("%s,%s,%s,%.3f,%.3f,%.3f,%.3f\n" % (
r["layer"], r["label"], r["net"] or "", r["x0"], r["y0"],
r["x1"], r["y1"]))
App.Console.PrintMessage("Rectangles exported to %s\n" % path)
except Exception as e:
App.Console.PrintWarning("CSV export failed: %s\n" % e)

doc.recompute()
if App.GuiUp:
import FreeCADGui as Gui
Gui.activeDocument().activeView().viewTop()
Gui.SendMsgToActiveView("ViewFit")

# ================================ REPORT ==================================
App.Console.PrintMessage(
"\n*** CONCEPTUAL FreeCAD VISUALIZATION - NOT a foundry-compliant layout ***\n")
print_table("Design-rule constraints and margins (PLACEHOLDER rules)", rule_rows)
App.Console.PrintMessage(
"\nPlaceholder-rule result: %d OK, %d NON-COMPLIANT (known concept "
"simplifications), %d FAIL; %d spacing/short violations.\n"
"Footprint %s x %s um (pad row %s um). A pass here is NOT foundry sign-off.\n"
% (n_ok, n_nc, n_fail, n_geom_viol, fmt(BW), fmt(BH),
fmt(pad_x[-1] - pad_x[0] + PAD)))
```

Listing 1. Python FreeCAD macro for generating the dimensioned 14-pad MOSFET test-module layout and its design-rule comparison tables.

Schematic:

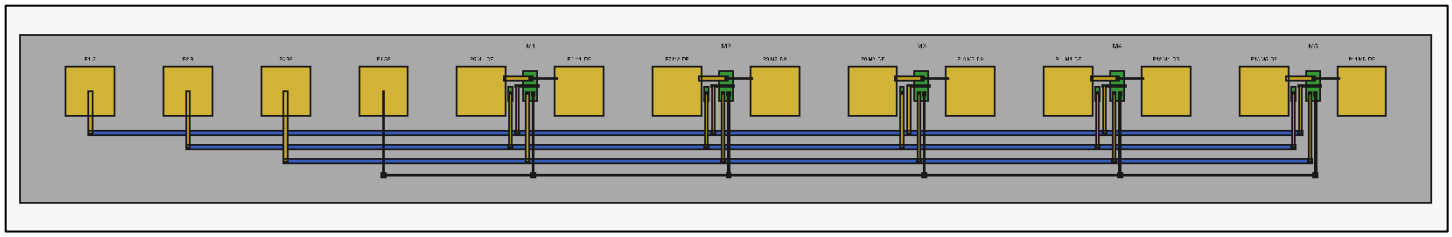


Figure 2. Conceptual top-view layout of the 14-pad MOSFET test module, illustrating the shared gate, body, and source connections and the dedicated drain force/sense pad pairs for five MOSFET test structures.

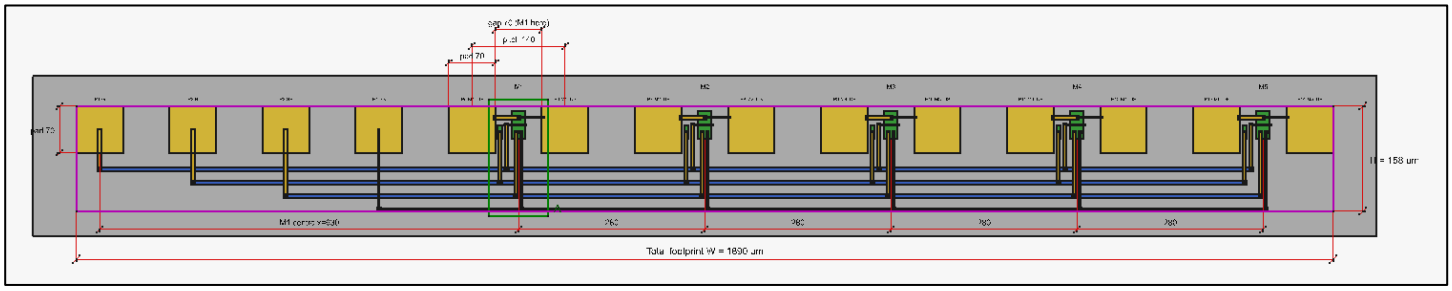


Figure 3. Dimensioned top view of the proposed five-MOSFET test module, showing pad locations, routing, device placement, interconnect dimensions, and the overall layout footprint.

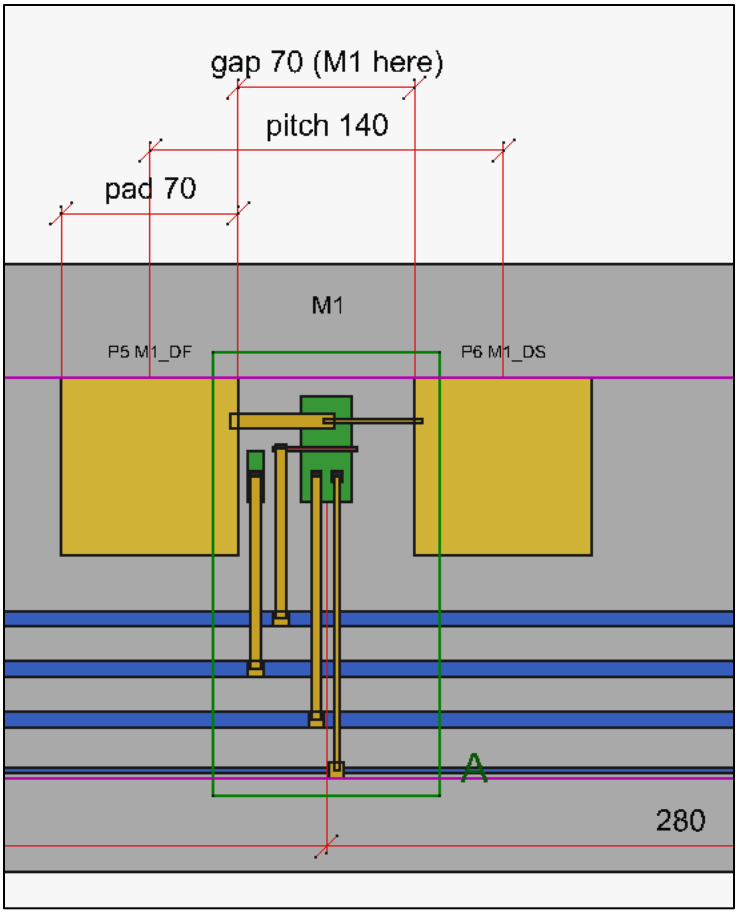


Figure 4. Enlarged view of Detail A, showing one MOSFET positioned within the 70 µm gap between adjacent drain pads, together with the local interconnect geometry and dimensional annotations.

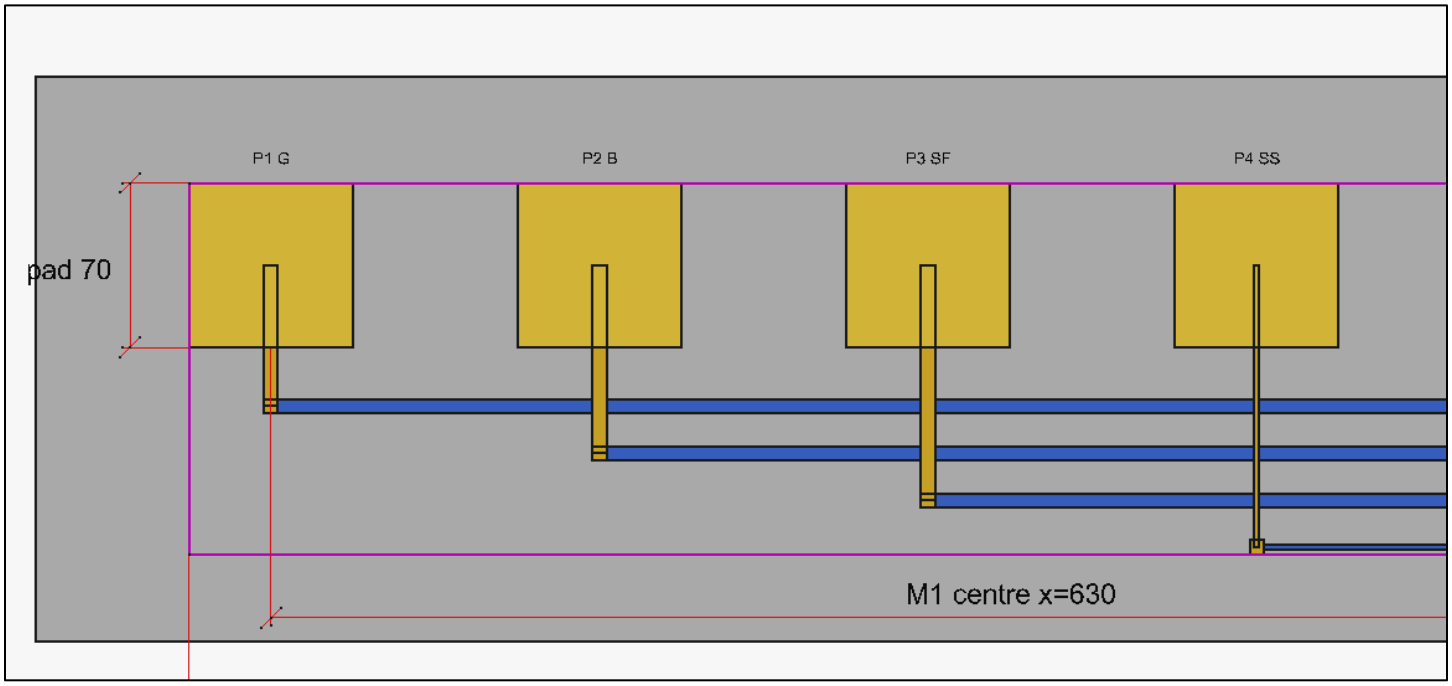


Figure 5. Close-up of the shared-pad routing region, illustrating pad placement, the 140 µm pad pitch, and the metal connections extending toward the MOSFET structures.

DESIGN-RULE CONSTRAINTS AND MARGINS (PLACEHOLDER RULES)

Rule / quantity	Layer	Drawn	Requirement	Margin	Status	Basis
Pad size	M2 pad	70.00 um	= 70.00 um	+0.00 um	OK	GIVEN
Pad size vs probe minimum	M2 pad	70.00 um	>= 50.00 um	+20.00 um	OK	ASSUMED rule
Pad pitch	M2 pad	140.00 um	= 140.00 um	+0.00 um	OK	GIVEN
Pad pitch vs probe minimum	M2 pad	140.00 um	>= 100.00 um	+40.00 um	OK	ASSUMED rule
Pad-to-pad spacing (gap)	M2 pad	70.00 um	>= 25.00 um	+45.00 um	OK	ASSUMED rule
M1 minimum trace width	M1	2.00 um	>= 0.50 um	+1.50 um	OK	ASSUMED rule
M1 spacing (different nets)	M1	14.00 um	>= 0.50 um	+13.50 um	OK	ASSUMED rule
M2 minimum trace width	M2	2.00 um	>= 0.60 um	+1.40 um	OK	ASSUMED rule
M2 spacing (different nets)	M2	4.00 um	>= 0.60 um	+3.40 um	OK	ASSUMED rule
Stub/trace to pad-edge clearance	M2	5.00 um	>= 0.60 um	+4.40 um	OK	ASSUMED rule
Widest non-pad metal (slotting)	M1/M2	6.00 um	<= 12.00 um	+6.00 um	OK	ASSUMED rule
M1 enclosure of via	M1	1.00 um	>= 0.20 um	+0.80 um	OK	ASSUMED rule
M2 enclosure of via	M2	1.00 um	>= 0.20 um	+0.80 um	OK	ASSUMED rule
Via size (single big via)	VIA	4.00 um	= 0.50 um	+3.50 um	NON-COMPLIANT	ASSUMED rule; needs via array
Contact size (single big contact)	CON	4.00 um	= 0.25 um	+3.75 um	NON-COMPLIANT	ASSUMED rule; needs contact array
Gate length L	POLY	2.00 um	>= 0.18 um	+1.82 um	OK	ASSUMED rule
Gate end-cap past active	POLY	2.00 um	>= 0.25 um	+1.75 um	OK	ASSUMED rule
Diffusion minimum width	DIFF	6.00 um	>= 0.30 um	+5.70 um	OK	ASSUMED rule
Body-tap to active spacing	DIFF	15.00 um	>= 0.40 um	+14.60 um	OK	ASSUMED rule
Poly to contact spacing	POLY/CON	7.00 um	>= 0.15 um	+6.85 um	OK	ASSUMED rule
Active to pad-edge (probe keep-out)	DIFF/M2	25.00 um	>= 10.00 um	+15.00 um	OK	ASSUMED rule
D-force trace current capacity	M2	12.00 mA	>= 5.00 mA	+7.00 mA	OK	CALC, assumed J
S-force stub current capacity	M2	8.00 mA	>= 5.00 mA	+3.00 mA	OK	CALC, assumed J
S-force bus current capacity	M1	6.00 mA	>= 5.00 mA	+1.00 mA	OK	CALC, assumed J
Device keep-in margin to pad band (bottom / top)	DIFF	21 / 7 um	-	-	INFO	CALC
Stub span inside gap (free: left / right)	M2	5 / 30 um	-	-	INFO	CALC
S-force path R, worst DUT (M5), est.	M1/M2	25.2 ohm = 126 mV @ 5 mA	-	-	INFO	removed by Kelvin sense
D-force path R, pad to drain, est.	M2	0.55 ohm = 2.7 mV @ 5 mA	-	-	INFO	removed by Kelvin sense
Pad-row extent	M2 pad	1890 um	-	-	INFO	CALC
Total footprint (all layout geometry)	all	1890 x 158 um ~ 0.2986 mm2	-	-	INFO	CALC
Assumed test current	-	5.0 mA	-	-	INFO	ASSUMED

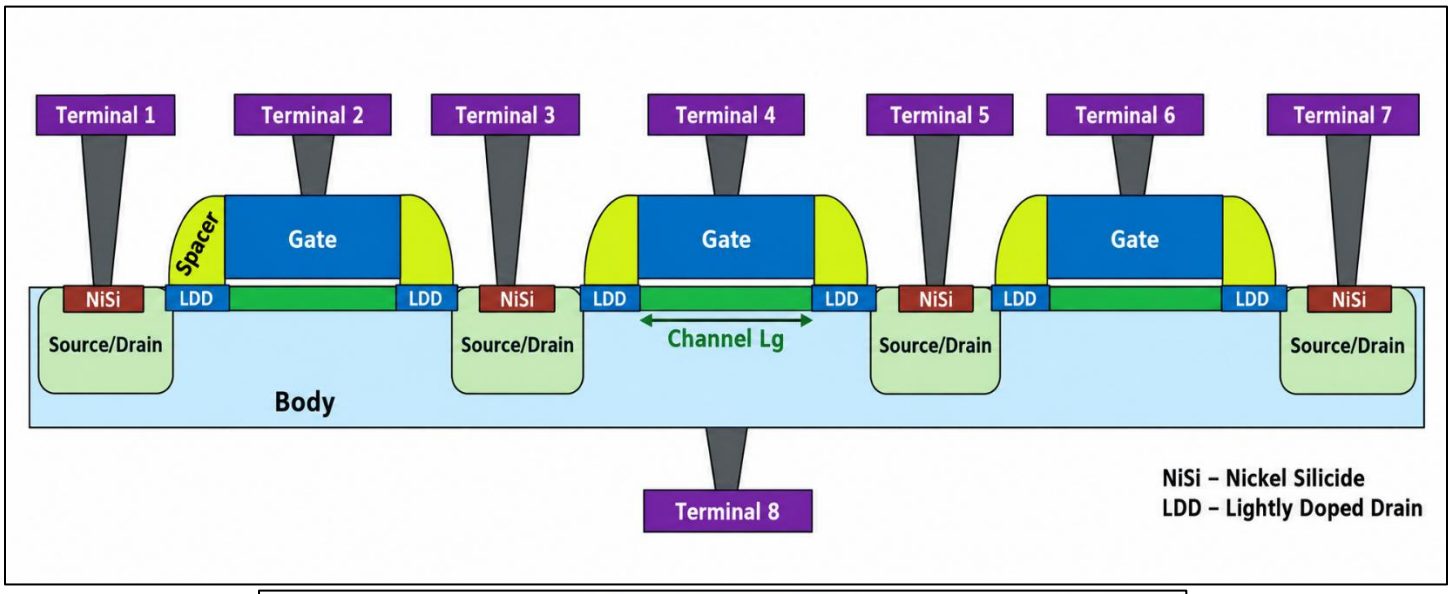
Table 1. Calculated design-rule margins for the conceptual FreeCAD MOSFET test-module layout, including pad dimensions, metal widths and spacings, via and contact dimensions, current-carrying capacity, estimated interconnect resistance, and total footprint. Rule values are illustrative placeholders and do not represent foundry sign-off.

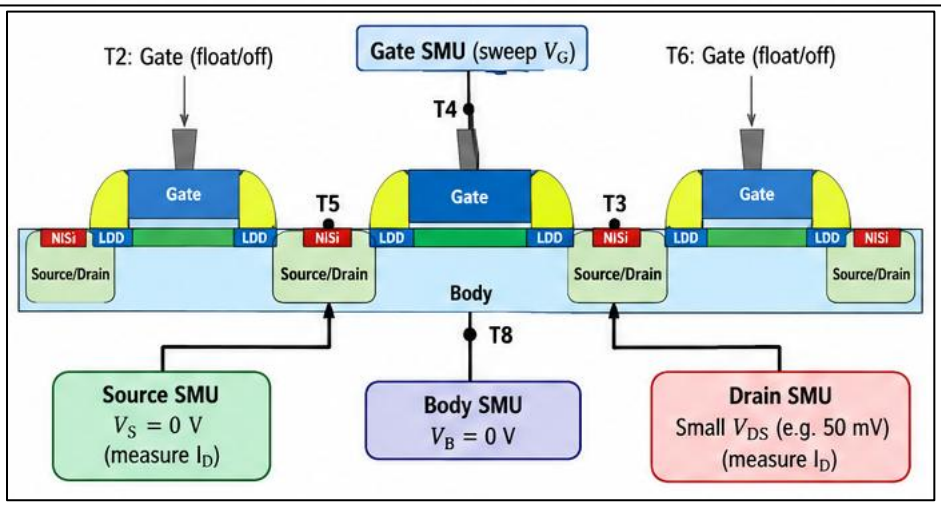
CONCEPTUAL FREECAD MODEL vs FOUNDRY-COMPLIANT LAYOUT

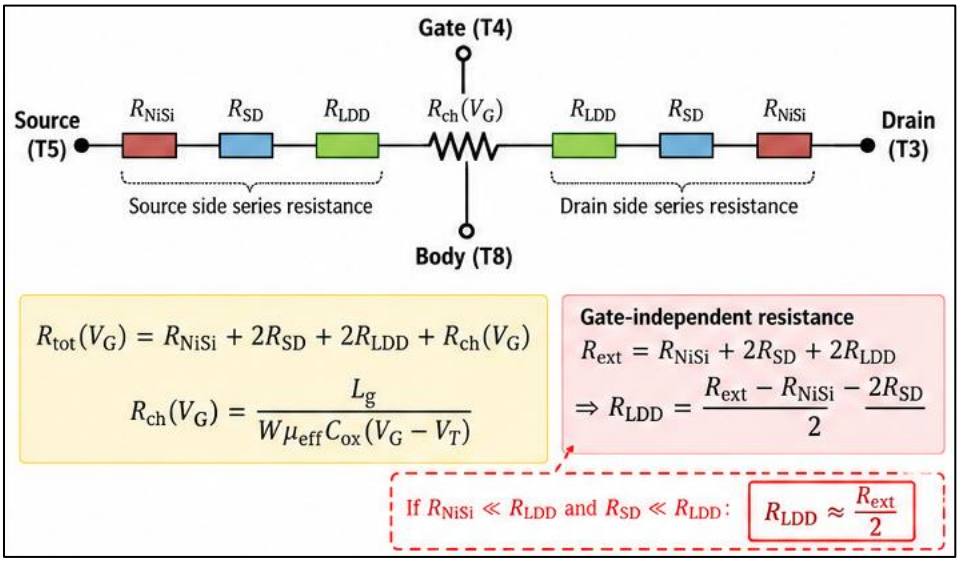
Aspect	This FreeCAD macro (CONCEPTUAL)	Foundry-compliant physical layout
Purpose	Topology + geometry visualization	Mask data (GDSII/OASIS) to fabricate
Layers	Generic SUB/DIFF/POLY/M1/VIA/M2 solids	PDK layer/datatype set, wells, implants, STI, pad opening
Rules	Placeholder values + simple spacing check	Full foundry DRC deck, clean (incl. density, antenna)
Contacts/vias	Single 4-6 um blocks	Exact-size contact/via arrays on the mfg grid
Isolation	None (one substrate block)	Wells/deep-NWELL as needed, guard rings, latch-up taps
Pads	70 um squares on M2	Pad cell: top-metal stack, passivation window, ESD/probe rules
Grid	Free-form um values	Manufacturing grid snapping, angle rules
Devices	Rectangles with W and L	PDK device cells with models, matching, parasitics
Sign-off	Visual review + this table	DRC + LVS + PEX + EM/IR in the PDK flow

Table 2. Comparison between the conceptual FreeCAD layout model and a foundry-compliant physical layout, highlighting differences in layer definitions, design-rule verification, contacts and vias, isolation structures, pad implementation, manufacturing-grid constraints, device models, and sign-off requirements.

Problem 2 — Solution:







Only the middle device (M2) is used for the LDD resistance measurement. A small drain–source voltage is applied across M2, and its gate voltage is swept to characterize the total resistance. The outer devices (M1 and M3) are not part of the measurement path and remain electrically inactive, with their gates and outer source/drain terminals

left floating. This configuration allows the LDD resistance of M2 to be extracted from the measured total resistance, provided the relevant parasitic resistances are accounted for.

Terminal assignments:

- T3 and T5: Source/drain terminals of M2, used to apply the drain–source bias and measure the drain current.
- T4: Gate terminal of M2, used to sweep the gate voltage.
- T2 and T6: Gate terminals of M1 and M3, respectively, left floating.
- T1 and T7: Outer source/drain terminals of M1 and M3, respectively, left floating.
- T8: Common body terminal, connected to ground or held at a specified body bias.

Assumptions

- R_{NiSi} (including contact and plug) is much smaller than R_{LDD}, so it can be neglected.
- The device is symmetric, so the source-side and drain-side LDD resistances are equal.
- The measurement is in the linear regime with a small V_{DS} (about 10–50 mV), so there is no field-dependent or saturation effect and no self-heating.
- Body (T8) is grounded, or at a fixed small reverse bias, so the junctions stay off and no current leaks through the body.
- The LDD under the spacer is only weakly gate-modulated. Its resistance saturates at high gate overdrive and does not depend on L_g.
- The channel follows the standard linear-region model, with mobility degradation handled by a fit.

Procedure

1. Force current through M2 only. Apply a small V_{DS} between T3 and T5. The ground is T5 and the body is T8. Measure I_D and calculate $R_{tot} = V_{DS} / I_D$.
2. Sweep the gate of M2 (T4) from just above threshold up to strong inversion, for example V_G − V_T = 0.3 V to 1.5 V or higher. Extract V_T beforehand from the linear-region I_D–V_G curve using the maximum-g_m extrapolation. The outer gates (T2, T6) and the outer terminals (T1, T7) float, so M1 and M3 are not part of the measurement path.
3. Model. The measured resistance contains two LDD segments in series with the channel:
$$R_{\text{tot}}(V_G) = R_{\text{NiSi}} + 2R_{\text{SD}} + 2R_{\text{LDD}} + R_{\text{ch}}(V_G)$$
$$R_{\text{ch}}(V_G) = \frac{L_g}{W \cdot \mu_{\text{eff}} \cdot C_{\text{ox}} \cdot (V_G - V_T)}$$
Since R_{NiSi} and R_{SD} are negligible,
$$R_{\text{tot}} \approx 2R_{\text{LDD}} + R_{\text{ch}}$$
4. Remove the channel term. Plot R_{tot} against $1/(V_G - V_T)$. At high overdrive this is a straight line, because R_{ch} is proportional to $1/(V_G - V_T)$. The intercept at $1/(V_G - V_T) \to 0$ gives $R_{\text{ext}} = 2R_{\text{LDD}}$. If mobility degradation bends the line, fit $R_{\text{tot}} = R_{\text{ext}} + \frac{a}{V_G - V_T} + b$ only over the high-overdrive region. Alternatively, use the Y-function.
5. Result: $R_{\text{LDD}} = \frac{R_{\text{ext}}}{2}$. The result can be multiplied by W to give a width-normalized value in $\Omega \cdot \mu$m.

Cross-checks

- Repeatability: repeat the measurement on M1 (T1–T3, gate T2) and M3 (T5–T7, gate T6). The three R_{LDD} values should agree. A spread indicates local variation, and any discrepancy would show a violated assumption.
- Reverse bias: repeat with a slightly reversed S/D bias to confirm that R_{LDD} does not depend on the bias polarity.
- Body bias: repeat at a small V_B. R_{ext} should stay constant, while V_T and the slope change.

Result:

$R_{\mathrm{LDD}} = \frac{1}{2}\lim_{V_{\mathrm{G}}\to\infty}\left[\frac{V_{\mathrm{DS}}}{I_{\mathrm{D}}}\right]$ across the M2 S/D (T3–T5), obtained from the intercept of R_{tot} versus $1/(V_{\mathrm{G}} - V_{\mathrm{T}})$.

Worked numerical example:

Parameter	Value
Drain-source voltage, V_{DS}	50 mV
Threshold voltage, V_T	0.50 V
Gate overdrive, $V_G - V_T$	0.50 V
Fitted intercept, R_{ext}	200 Ω
R_{NiSi}	10 Ω
R_{SD}, per side	20 Ω

Step 1 — Calculate total resistance at one operating point.
For the illustrative model $R_{\mathrm{tot}} = 200 + 200/(V_G - V_T)$,

$$R_{\mathrm{tot}} = 200 + \frac{200}{0.50} = 600\,\Omega$$

Step 2 — Calculate drain current.

$$I_D = \frac{V_{DS}}{R_{\mathrm{tot}}} = \frac{0.050}{600} = 83.33\,\mu\mathrm{A}$$

Step 3 — Extract LDD resistance from the intercept.

$$R_{\mathrm{LDD}} = \frac{R_{\mathrm{ext}} - R_{\mathrm{NiSi}} - 2R_{\mathrm{SD}}}{2}$$

Substituting the values:

$$R_{\mathrm{LDD}} = \frac{200 - 10 - 2(20)}{2}$$
$$= \frac{150}{2}$$
$$= \boxed{75\,\Omega}$$

If both R_{NiSi} and R_{SD} are negligible, the same intercept gives $R_{\mathrm{LDD}} \approx 200/2 = 100\,\Omega$.

ldd_extraction_plot.py

```
"""
Illustrative MOSFET LDD resistance extraction.
The data are synthetic examples, NOT experimental measurements.

Run:
    py -m pip install numpy matplotlib
    py ldd_resistance_extraction.py
"""

import numpy as np
import matplotlib.pyplot as plt

# 1. Measurement/model assumptions
VDS = 0.050       # Drain-source voltage: 50 mV
VT = 0.50         # Assumed threshold voltage (V)
R_NiSi = 10.0     # Illustrative NiSi resistance (ohm)
R_SD = 20.0       # Illustrative source/drain resistance per side (ohm)

# Synthetic example model
Rext_true = 200.0  # Expected resistance intercept (ohm)
k_true = 200.0     # Expected slope (ohm*V)

# 2. Gate-voltage sweep
overdrive = np.array([
    0.30, 0.35, 0.40, 0.50, 0.60,
    0.80, 1.00, 1.20, 1.50
])
```

```
VG = VT + overdrive
x = 1.0 / (VG - VT)  # Inverse gate overdrive (1/V)

# Synthetic deviations from the ideal straight line
noise_ohm = np.array([
    2.0, -3.0, 1.0, -2.0, 3.0,
    -1.0, 2.0, -2.0, 0.0
])

# Total resistance (synthetic data)
Rtot = Rext_true + k_true * x + noise_ohm

# 3. Linear regression: Rtot = intercept + slope*x
slope, intercept = np.polyfit(x, Rtot, 1)

Rext = intercept
RLDD = (Rext - R_NiSi - 2.0 * R_SD) / 2.0

# 4. Example operating point
example_overdrive = 0.50
example_Rtot = Rext_true + k_true / example_overdrive
example_ID = VDS / example_Rtot

# 5. Print results
print("ILLUSTRATIVE EXAMPLE: synthetic data, not measurements")
print(f"VDS = {VDS * 1000:.1f} mV")
print(f"Gate overdrive = {example_overdrive:.2f} V")
print(f"Example total resistance = {example_Rtot:.2f} ohm")
print(f"Example drain current = {example_ID * 1e6:.2f} microamp")
print()
print(f"Fitted slope k = {slope:.3f} ohm*V")
print(f"Fitted intercept Rext = {Rext:.3f} ohm")
print(f"Assumed R_NiSi = {R_NiSi:.2f} ohm")
print(f"Assumed R_SD per side = {R_SD:.2f} ohm")
print(f"Extracted RLDD = {RLDD:.3f} ohm")

# 6. Plot the data and fitted line
x_fit = np.linspace(0.0, max(x) * 1.08, 250)
R_fit = intercept + slope * x_fit

fig, ax = plt.subplots(figsize=(9, 6))

ax.scatter(x, Rtot, label="Illustrative synthetic data")
ax.plot(x_fit, R_fit, label="Linear fit", linewidth=2)

ax.axhline(
    intercept,
    linestyle="--",
    linewidth=1.2,
    label=f"Y-intercept = {intercept:.1f} ohm"
)

ax.scatter([0], [intercept], zorder=4)

ax.annotate(
    f"Rext = {intercept:.1f} ohm",
    xy=(0, intercept),
    xytext=(0.65, intercept + 70),
    arrowprops={"arrowstyle": "->"}
)

ax.set_xlabel(r"$1/(V_G - V_T)$ (V$^{-1}$)")
ax.set_ylabel(r"$R_{\mathrm{tot}}=V_{DS}/I_D$ ($\Omega$)")
ax.set_title("Illustrative LDD Resistance Extraction")
ax.grid(True, alpha=0.3)
ax.legend()

# Robust automatic figure layout
fig.set_constrained_layout(True)
```

```
fig.savefig(
    "ldd_resistance_extraction_example.png",
    dpi=200,
    bbox_inches="tight"
)

print("\nPlot saved as ldd_resistance_extraction_example.png")
plt.show()
```

Listing 2. Python implementation of the illustrative LDD-resistance extraction method using NumPy linear regression and Matplotlib plotting.

```
PS C:\Users\Lenovo\Micron Open Day at IIT Bomabay\problem 2> py ldd_resistance_extraction.py
ILLUSTRATIVE EXAMPLE: synthetic data, not measurements
VDS = 50.0 mV
Gate overdrive = 0.50 V
Example total resistance = 600.00 ohm
Example drain current = 83.33 microamp

Fitted slope k = 200.095 ohm*V
Fitted intercept Rext = 199.831 ohm
Assumed R_NiSi = 10.00 ohm
Assumed R_SD per side = 20.00 ohm
Extracted RLDD = 74.915 ohm

Plot saved as ldd_resistance_extraction_example.png
```

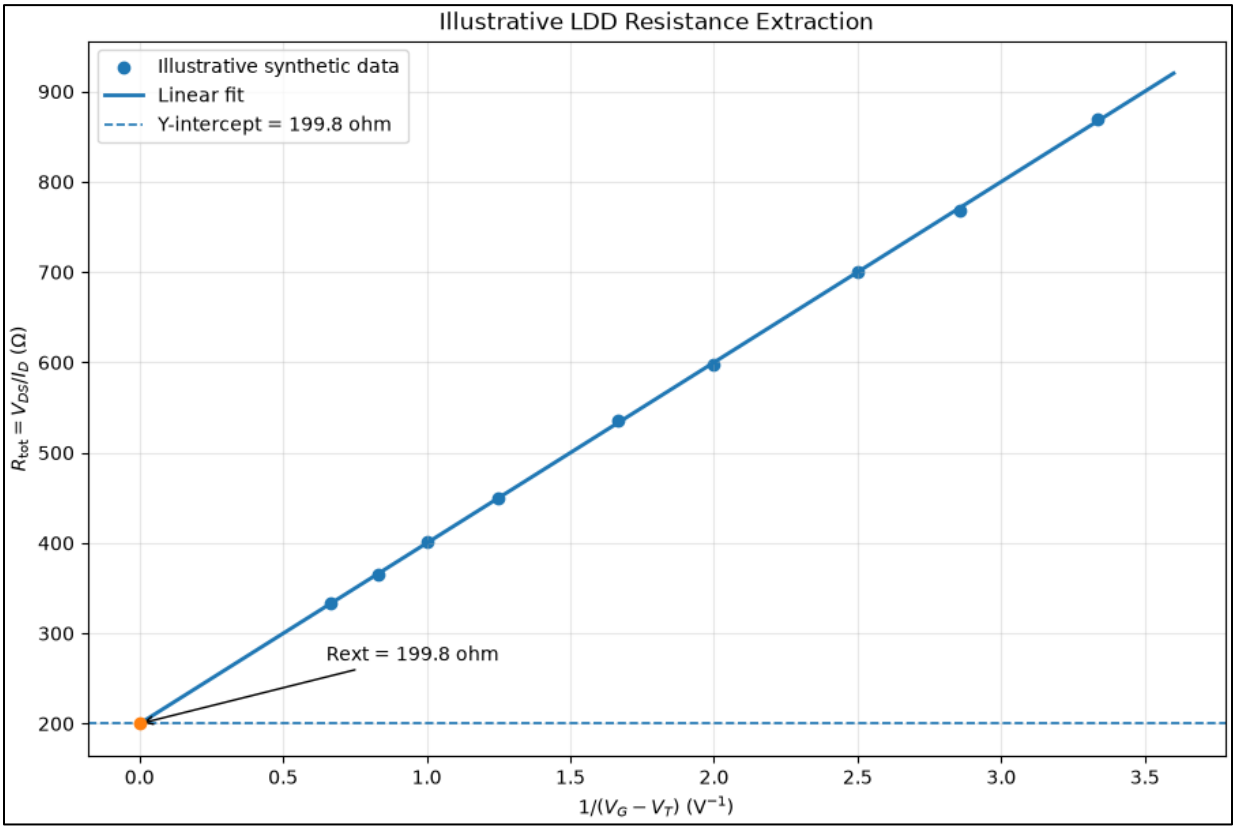


Figure 6. Illustrative LDD-resistance extraction plot showing total MOSFET resistance, $R_{tot} = V_{DS}/I_D$, versus inverse gate overdrive, $1/(V_G - V_T)$. The fitted straight line is extrapolated to zero inverse overdrive to estimate the gate-independent resistance intercept, R_{ext}, from which the LDD resistance is extracted after accounting for relevant parasitic resistances.

ldd_analysis.py

```
"""
LDD Resistance Characterization and MOSFET I-V Analysis
-------------------------------------------------------
Input:
    data/measurements.csv

Output:
    results/ldd_results.csv
    results/transfer_characteristics.png
    results/output_characteristics.png
    results/ldd_resistance_fit.png

IMPORTANT:
The generated dataset is illustrative, not measured silicon data.
A valid LDD resistance extraction requires the voltage drop across
the actual LDD segment, measured or reliably estimated.
"""

from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# ---------------------------------------------------------
# 1. File paths
# ---------------------------------------------------------

PROJECT_DIR = Path(__file__).resolve().parent
DATA_FILE = PROJECT_DIR / "data" / "measurements.csv"
RESULTS_DIR = PROJECT_DIR / "results"

RESULTS_DIR.mkdir(parents=True, exist_ok=True)

# ---------------------------------------------------------
# 2. Load and validate CSV data
# ---------------------------------------------------------

REQUIRED_COLUMNS = [
    "sweep_type",
    "vg_V",
    "vds_V",
    "id_A",
    "v_ldd_start_V",
    "v_ldd_end_V",
]

NUMERIC_COLUMNS = [
    "vg_V",
    "vds_V",
    "id_A",
    "v_ldd_start_V",
    "v_ldd_end_V",
]

def load_measurements(csv_path):
    """Load measurements and validate the required columns."""

    if not csv_path.exists():
        raise FileNotFoundError(
            f"\nCSV file not found:\n{csv_path}\n\n"
            "Create a data folder beside this script and place "
            "measurements.csv inside it."
        )

    df = pd.read_csv(csv_path)

    # Remove accidental spaces around column names.
    df.columns = df.columns.str.strip()

    missing_columns = [
        column
        for column in REQUIRED_COLUMNS
        if column not in df.columns
    ]
```

```
    if missing_columns:
        raise ValueError(
            f"CSV is missing columns: {missing_columns}\n"
            f"Columns found: {list(df.columns)}"
        )

    # Standardize sweep labels.
    df["sweep_type"] = (
        df["sweep_type"]
        .astype("string")
        .str.strip()
        .str.lower()
    )

    valid_sweeps = {"transfer", "output", "ldd"}

    unknown_sweeps = set(df["sweep_type"].dropna().unique()) - valid_sweeps

    if unknown_sweeps:
        raise ValueError(
            f"Unknown sweep types: {unknown_sweeps}. "
            "Use transfer, output, or ldd."
        )

    if df["sweep_type"].isna().any():
        raise ValueError("Some rows have a missing sweep_type.")

    # Convert measurement columns to numeric values.
    for column in NUMERIC_COLUMNS:
        df[column] = pd.to_numeric(df[column], errors="coerce")

    # Every row needs these three terminal measurements.
    base_columns = ["vg_V", "vds_V", "id_A"]

    invalid_base = df[base_columns].isna().any(axis=1)

    if invalid_base.any():
        bad_rows = (df.index[invalid_base] + 2).tolist()
        raise ValueError(
            "Missing or invalid gate/drain measurements "
            f"in CSV row(s): {bad_rows}"
        )

    # Reject infinite values in the required terminal measurements.
    if not np.isfinite(df[base_columns].to_numpy(dtype=float)).all():
        raise ValueError(
            "Gate voltage, drain voltage, and drain current "
            "must be finite numbers."
        )

    # LDD endpoint voltages are required only for LDD rows.
    ldd_mask = df["sweep_type"].eq("ldd")
    ldd_columns = ["v_ldd_start_V", "v_ldd_end_V"]

    if df.loc[ldd_mask, ldd_columns].isna().any().any():
        bad_rows = (
            df.loc[ldd_mask]
            .index[
                df.loc[ldd_mask, ldd_columns].isna().any(axis=1)
            ] + 2
        ).tolist()

        raise ValueError(
            "LDD rows must contain both endpoint voltages. "
            f"Check CSV row(s): {bad_rows}"
        )

    if ldd_mask.any():
        ldd_values = df.loc[ldd_mask, ldd_columns].to_numpy(dtype=float)

        if not np.isfinite(ldd_values).all():
            raise ValueError(
                "LDD endpoint voltages must be finite numbers."
            )

    if (df["id_A"] < 0).any():
        raise ValueError(
            "This script expects non-negative drain current. "
            "Check the current sign convention in your CSV."
```

```
        )

    if df.empty:
        raise ValueError("The CSV contains no measurement rows.")

    print(f"Loaded {len(df)} measurement rows.")
    print(f"Input file: {csv_path}")

    return df

# ------------------------------------------------------------
# 3. Calculate LDD voltage drop and pointwise resistance
# ------------------------------------------------------------

def calculate_ldd_resistance(df):
    """Calculate resistance only for rows marked as LDD."""

    results = df.copy()

    results["delta_v_ldd_V"] = np.nan
    results["r_ldd_ohm"] = np.nan

    ldd_mask = results["sweep_type"].eq("ldd")

    # Keep the signed voltage difference.
    results.loc[ldd_mask, "delta_v_ldd_V"] = (
        results.loc[ldd_mask, "v_ldd_start_V"]
        - results.loc[ldd_mask, "v_ldd_end_V"]
    )

    # R = |V| / |I|. Resistance is undefined at zero current.
    valid_current = ldd_mask & results["id_A"].ne(0)

    results.loc[valid_current, "r_ldd_ohm"] = (
        results.loc[valid_current, "delta_v_ldd_V"].abs()
        / results.loc[valid_current, "id_A"].abs()
    )

    output_csv = RESULTS_DIR / "ldd_results.csv"
    results.to_csv(output_csv, index=False)

    print(f"Detailed results saved to: {output_csv}")

    return results

# ------------------------------------------------------------
# 4. Plot transfer characteristics: ID versus VG
# ------------------------------------------------------------

def plot_transfer_characteristics(df):
    transfer = df[df["sweep_type"] == "transfer"].copy()

    if transfer.empty:
        print("No transfer data found. Skipping transfer plot.")
        return

    plt.figure(figsize=(8, 5))

    for vds, group in transfer.groupby("vds_V"):
        group = group.sort_values("vg_V")

        plt.plot(
            group["vg_V"],
            group["id_A"] * 1e6,
            marker="o",
            markersize=4,
            label=f"VDS = {vds:g} V",
        )

    plt.xlabel("Gate-to-source voltage VG (V)")
    plt.ylabel("Drain current ID (µA)")
    plt.title("MOSFET Transfer Characteristics")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()

    output_file = RESULTS_DIR / "transfer_characteristics.png"
    plt.savefig(output_file, dpi=200)
    plt.close()
```

```
    print(f"Transfer plot saved to: {output_file}")

# ------------------------------------------------------------
# 5. Plot output characteristics: ID versus VDS
# ------------------------------------------------------------

def plot_output_characteristics(df):
    output = df[df["sweep_type"] == "output"].copy()

    if output.empty:
        print("No output data found. Skipping output plot.")
        return

    plt.figure(figsize=(8, 5))

    for vg, group in output.groupby("vg_V"):
        group = group.sort_values("vds_V")

        plt.plot(
            group["vds_V"],
            group["id_A"] * 1e6,
            marker="o",
            markersize=3,
            label=f"VG = {vg:g} V",
        )

    plt.xlabel("Drain-to-source voltage VDS (V)")
    plt.ylabel("Drain current ID (μA)")
    plt.title("MOSFET Output Characteristics")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()

    output_file = RESULTS_DIR / "output_characteristics.png"
    plt.savefig(output_file, dpi=200)
    plt.close()

    print(f"Output plot saved to: {output_file}")

# ------------------------------------------------------------
# 6. Fit LDD voltage drop against current
# ------------------------------------------------------------

def fit_ldd_resistance(df):
    """
    Fit |V_LDD,start - V_LDD,end| = R * |ID| + intercept.

    The fitted slope estimates resistance only if the measured
    voltage difference corresponds to the actual LDD segment.
    """

    ldd = df[df["sweep_type"] == "ldd"].copy()

    if ldd.empty:
        print("No LDD rows found. Cannot fit LDD resistance.")
        return None

    ldd["delta_v_abs_V"] = (
        ldd["v_ldd_start_V"] - ldd["v_ldd_end_V"]
    ).abs()

    ldd["current_abs_A"] = ldd["id_A"].abs()

    # A zero-current point cannot be used to estimate V/I.
    fit_data = ldd[
        (ldd["current_abs_A"] > 0)
        & np.isfinite(ldd["current_abs_A"])
        & np.isfinite(ldd["delta_v_abs_V"])
    ].copy()

    if len(fit_data) < 2:
        print("At least two valid nonzero-current LDD points are needed.")
        return None

    current = fit_data["current_abs_A"].to_numpy()
    voltage = fit_data["delta_v_abs_V"].to_numpy()

    # Linear model: voltage = slope * current + intercept.
```

```
    slope, intercept = np.polyfit(current, voltage, 1)

    predicted_voltage = slope * current + intercept

    ss_res = np.sum((voltage - predicted_voltage) ** 2)
    ss_tot = np.sum((voltage - np.mean(voltage)) ** 2)

    r_squared = (
        1.0 - ss_res / ss_tot
        if ss_tot > 0
        else float("nan")
    )

    print("\n========== LDD RESISTANCE ANALYSIS ==========")
    print(f"Valid LDD data points : {len(fit_data)}")
    print(f"Fitted resistance     : {slope:.3f} ohms")
    print(f"Fitted intercept      : {intercept:.6e} V")
    print(f"R-squared             : {r_squared:.6f}")
    print("Model: |V_LDD| = R_LDD * |ID| + intercept")
    print("============================================\n")

    # Plot data and fitted line.
    plt.figure(figsize=(8, 5))

    plt.scatter(
        current * 1e6,
        voltage * 1e3,
        label="LDD data",
        marker="o",
        s=45,
    )

    current_line = np.linspace(current.min(), current.max(), 200)
    voltage_line = slope * current_line + intercept

    plt.plot(
        current_line * 1e6,
        voltage_line * 1e3,
        label=f"Linear fit: R = {slope:.2f} Ω",
    )

    plt.xlabel("Absolute drain current |ID| (µA)")
    plt.ylabel("Absolute LDD voltage drop (mV)")
    plt.title("LDD Resistance Extraction")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()

    output_file = RESULTS_DIR / "ldd_resistance_fit.png"
    plt.savefig(output_file, dpi=200)
    plt.close()

    print(f"LDD fit plot saved to: {output_file}")

    return {
        "resistance_ohm": slope,
        "intercept_V": intercept,
        "r_squared": r_squared,
        "number_of_points": len(fit_data),
    }

# ------------------------------------------------------------
# 7. Main program
# ------------------------------------------------------------

def main():
    print("\nMOSFET LDD Resistance Characterization")
    print("--------------------------------------")

    df = load_measurements(DATA_FILE)

    print("\nRows by sweep type:")
    print(df["sweep_type"].value_counts().to_string())

    results = calculate_ldd_resistance(df)

    plot_transfer_characteristics(df)
    plot_output_characteristics(df)
```

```
    fit = fit_ldd_resistance(df)

    print("\nAnalysis completed.")
    print(f"All results are in: {RESULTS_DIR}")

    if fit is not None:
        print(
            "\nEstimated LDD resistance: "
            f"{fit['resistance_ohm']:.3f} ohms"
        )

    print(
        "\nReminder: if the input CSV contains synthetic data, "
        "the extracted value is only a software demonstration."
    )

if __name__ == "__main__":
    try:
        main()
    except (ValueError, FileNotFoundError, pd.errors.ParserError) as error:
        print(f"\nERROR: {error}")
        raise SystemExit(1)
```

measurements.csv

```
sweep_type,vg_V,vds_V,id_A,v_ldd_start_V,v_ldd_end_V
transfer,0,0.05,1.13475894e-09,,
transfer,0.1,0.05,1.470354917e-09,,
transfer,0.2,0.05,2.641699972e-09,,
transfer,0.3,0.05,6.730095937e-09,,
transfer,0.4,0.05,2.1e-08,,
transfer,0.5,0.05,4.19052463e-06,,
transfer,0.6,0.05,1.316525277e-05,,
transfer,0.7,0.05,2.603069721e-05,,
transfer,0.8,0.05,4.232423054e-05,,
transfer,0.9,0.05,6.175722067e-05,,
transfer,1,0.05,8.412418285e-05,,
transfer,1.1,0.05,0.0001092678278,,
transfer,1.2,0.05,0.0001370620288,,
output,0.6,0,0,,
output,0.6,0.1,6.015431618e-06,,
output,0.6,0.2,9.953254829e-06,,
output,0.6,0.3,1.187630162e-05,,
output,0.6,0.4,1.269884304e-05,,
output,0.6,0.5,1.304753784e-05,,
output,0.6,0.6,1.321076368e-05,,
output,0.6,0.7,1.330399436e-05,,
output,0.6,0.8,1.33711668e-05,,
output,0.6,0.9,1.3428687e-05,,
output,0.6,1,1.348263861e-05,,
output,0.8,0,0,,
output,0.8,0.1,1.035827824e-05,,
output,0.8,0.2,1.962206031e-05,,
output,0.8,0.3,2.707622283e-05,,
output,0.8,0.4,3.259487289e-05,,
output,0.8,0.5,3.644796152e-05,,
output,0.8,0.6,3.904376188e-05,,
output,0.8,0.7,4.076505237e-05,,
output,0.8,0.8,4.190840547e-05,,
output,0.8,0.9,4.268174704e-05,,
output,0.8,1,4.322278287e-05,,
output,1,0,0,,
output,1,0.1,1.391471131e-05,,
output,1,0.2,2.719855532e-05,,
output,1,0.3,3.924819846e-05,,
output,1,0.4,4.969213579e-05,,
output,1,0.5,5.84034285e-05,,
output,1,0.6,6.545015521e-05,,
output,1,0.7,7.102086186e-05,,
output,1,0.8,7.535582884e-05,,
output,1,0.9,7.869852534e-05,,
output,1,1,8.126831757e-05,,
output,1.2,0,0,,
output,1.2,0.1,1.708728093e-05,,
output,1.2,0.2,3.378822555e-05,,
output,1.2,0.3,4.963406682e-05,,
output,1.2,0.4,6.425823235e-05,,
```

```
output,1.2,0.5,7.742171604e-05,,
output,1.2,0.6,8.901404079e-05,,
output,1.2,0.7,9.903541161e-05,,
output,1.2,0.8,0.0001075687876,,
output,1.2,0.9,0.0001147500395,,
output,1.2,1,0.0001207417328,,
ldd,1,0.00485,5e-05,0.01485,0.01
ldd,1,0.0076,7.5e-05,0.0186,0.011
ldd,1,0.00995,0.0001,0.02195,0.012
ldd,1,0.01262,0.000125,0.02562,0.013
ldd,1,0.01492,0.00015,0.02892,0.014
ldd,1,0.01756,0.000175,0.03256,0.015
ldd,1,0.02,0.0002,0.036,0.016
```

```
PS C:\Users\Lenovo\Micron Open Day at IIT Bomabay\problem 2> py ldd_analysis.py

MOSFET LDD Resistance Characterization
--------------------------------------
Loaded 64 measurement rows.
Input file: C:\Users\Lenovo\Micron Open Day at IIT Bomabay\problem 2\data\measurements.csv

Rows by sweep type:
sweep_type
output      44
transfer    13
ldd          7
Detailed results saved to: C:\Users\Lenovo\Micron Open Day at IIT Bomabay\problem 2\results\ldd_results.csv
Transfer plot saved to: C:\Users\Lenovo\Micron Open Day at IIT Bomabay\problem 2\results\transfer_characteristics.png
Output plot saved to: C:\Users\Lenovo\Micron Open Day at IIT Bomabay\problem 2\results\output_characteristics.png

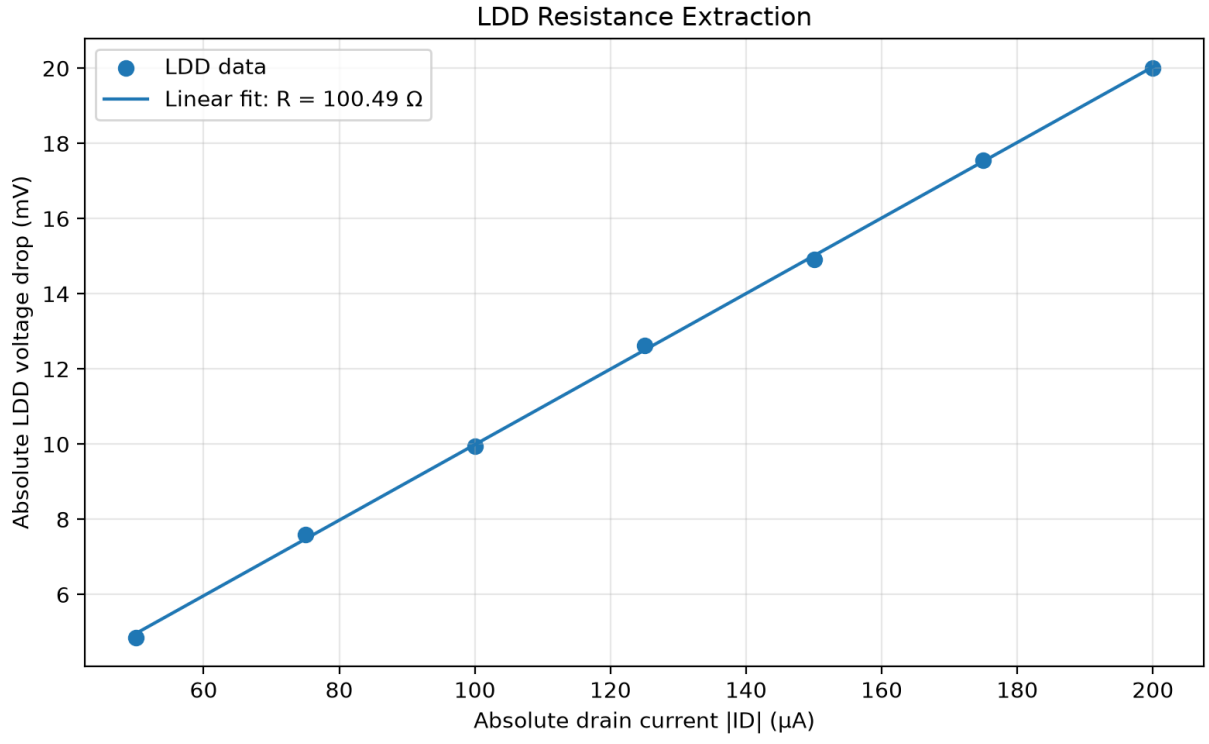
========== LDD RESISTANCE ANALYSIS ==========
Valid LDD data points : 7
Fitted resistance     : 100.486 ohms
Fitted intercept      : -6.071429e-05 V
R-squared             : 0.999687
Model: |V_LDD| = R_LDD * |ID| + intercept
=============================================

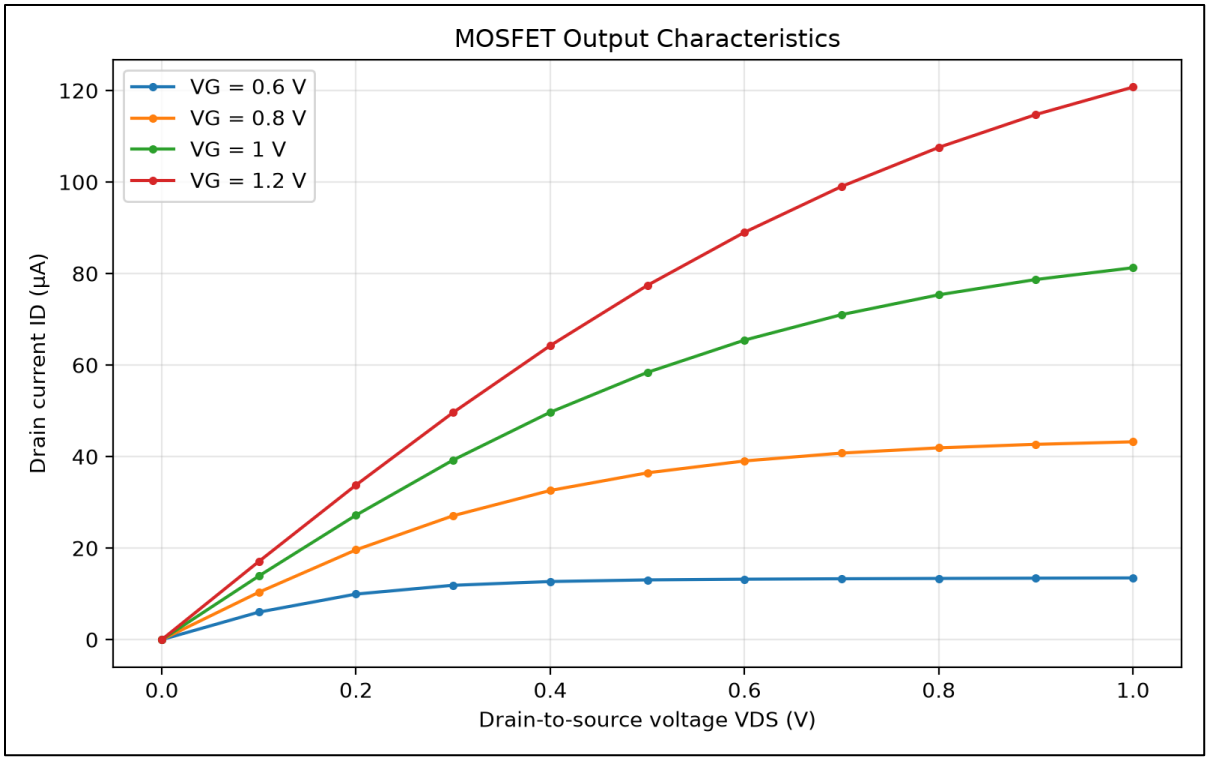
LDD fit plot saved to: C:\Users\Lenovo\Micron Open Day at IIT Bomabay\problem 2\results\ldd_resistance_fit.png

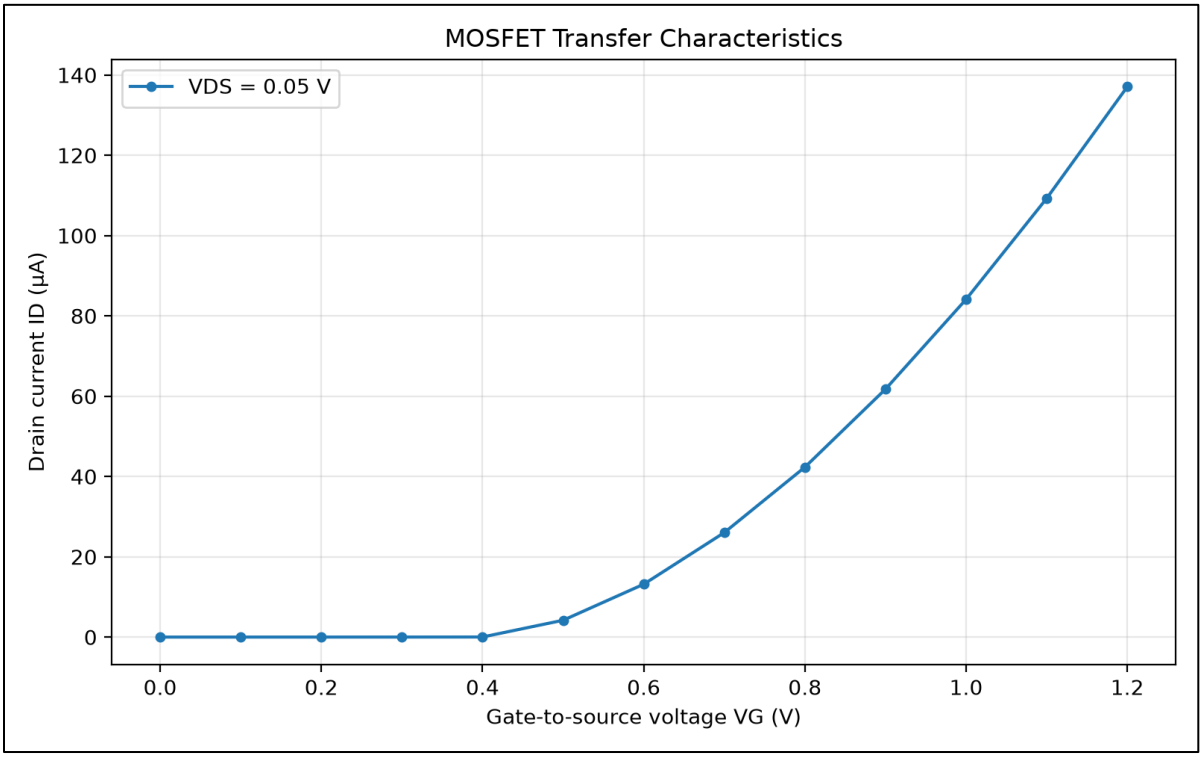
Analysis completed.
All results are in: C:\Users\Lenovo\Micron Open Day at IIT Bomabay\problem 2\results

Estimated LDD resistance: 100.486 ohms

Reminder: if the input CSV contains synthetic data, the extracted value is only a software demonstration.
```




MOSFET Output Characteristics
VG = 0.6 V
VG = 0.8 V
VG = 1 V
VG = 1.2 V
Drain current ID (μA)
Drain-to-source voltage VDS (V)
0
20
40
60
80
100
120
0.0
0.2
0.4
0.6
0.8
1.0


MOSFET Transfer Characteristics
VDS = 0.05 V
Drain current ID (μA)
Gate-to-source voltage VG (V)
0
20
40
60
80
100
120
140
0.0
0.2
0.4
0.6
0.8
1.0
1.2

Schematic Macro code for freeCAD tool:

```
# -*- coding: utf-8 -*-
# FreeCAD macro: 3 identical MOSFETs + measurement wiring for LDD extraction
#
# Measurement (middle device M2, between T3 and T5):
# T3 : S/D of M2 -> SMU1 (force small V_DS, measure I_D) [red wire]
# T4 : gate of M2 -> SMU2 (sweep V_G, V_G - V_T = 0.3 ... 1.5 V) [blue wire]
# T5 : S/D of M2 -> GND [black wire]
# T8 : body -> GND [black wire]
# T1, T2, T6, T7 -> floating (open stubs) [gray]
#
# Run: Macro > Macros... > Execute (or paste in the Python console)

import FreeCAD as App
import Part

V = App.Vector

# ---------------- Device parameters (edit freely) ----------------
Lg = 40.0
Lsp = 20.0 # spacer length = LDD length
SDw = 70.0
xj = 35.0
xj_ldd = 12.0
tox = 2.0
Hg = 50.0
tNiSi = 6.0
NiSi_in = 8.0
W = 120.0
bodyT = 80.0
margin = 20.0
plugW = 14.0
plugH = 70.0
padW = 50.0
padT = 10.0
padStag = 6.0

# ---------------- Wiring parameters ----------------
wr = 2.0 # wire radius
stubH = 30.0 # height of floating stubs
instW = 70.0 # instrument box size (X)
instH = 30.0 # instrument box height (Z)

doc = App.newDocument("Three_MOSFET_LDD_Measurement")

C = {
"body": (0.75, 0.85, 0.95), "sd": (0.65, 0.85, 0.60),
"ldd": (0.20, 0.55, 0.95), "chan": (0.10, 0.70, 0.35),
"ox": (0.95, 0.95, 0.95), "gate": (0.15, 0.35, 0.85),
"spacer": (0.85, 0.90, 0.20), "nisi": (0.80, 0.30, 0.30),
"plug": (0.30, 0.30, 0.30), "pad": (0.55, 0.20, 0.70),
"wD": (0.90, 0.10, 0.10), "wG": (0.10, 0.30, 0.95),
"wGND": (0.05, 0.05, 0.05), "wF": (0.60, 0.60, 0.60),
"inst": (1.00, 0.75, 0.20),
}

def add(name, shape, ckey, transp=0):
obj = doc.addObject("Part::Feature", name)
obj.Shape = shape
if App.GuiUp:
obj.ViewObject.ShapeColor = C[ckey]
obj.ViewObject.Transparency = transp
return obj

def box(x, z, lx, lz, y=0.0, ly=None):
return Part.makeBox(lx, W if ly is None else ly, lz, V(x, y, z))

def label(text, x, y, z):
"""Text label in the 3D view (GUI only; silently skipped otherwise)."""
if not App.GuiUp:
return
try:
import Draft
Draft.make_text(text if isinstance(text, list) else [text], V(x, y, z))
except Exception as e:
App.Console.PrintWarning("Label skipped: %s\n" % e)

# ---------------- Device geometry ----------------
```

```
n_dev = 3
pitch = SDw + 2 * Lsp + Lg
total = n_dev * pitch + SDw
bodyL = total + 2 * margin
add("Body_T8", box(0, -bodyT, bodyL, bodyT), "body", 60)

x0 = margin
pads = {} # terminal -> (x_centre, z_top_of_pad)
zTop_gate = tox + Hg

def pad_and_plug(tnum, xc, z_base, stag):
yc = W / 2.0
add("Plug_T%d" % tnum,
Part.makeBox(plugW, W * 0.6, plugH, V(xc - plugW / 2, yc - W * 0.3, z_base)),
"plug")
zpad = z_base + plugH + stag
add("Terminal_%d" % tnum,
Part.makeBox(padW, W * 0.8, padT, V(xc - padW / 2, yc - W * 0.4, zpad)),
"pad")
pads[tnum] = (xc, zpad + padT)

stag = 0.0
for i in range(n_dev + 1):
xs = x0 + i * pitch
add("SD_%d" % (i + 1), box(xs, -xj, SDw, xj), "sd", 30)
add("NiSi_%d" % (i + 1), box(xs + NiSi_in, 0, SDw - 2 * NiSi_in, tNiSi), "nisi")
pad_and_plug(2 * i + 1, xs + SDw / 2.0, tNiSi, stag)
stag += padStag
if i == n_dev:
break

xl = xs + SDw
xc = xl + Lsp
xr = xc + Lg
add("LDD_L_%d" % (i + 1), box(xl, -xj_ldd, Lsp, xj_ldd), "ldd")
add("Channel_%d" % (i + 1), box(xc, -xj_ldd * 0.5, Lg, xj_ldd * 0.5), "chan")
add("LDD_R_%d" % (i + 1), box(xr, -xj_ldd, Lsp, xj_ldd), "ldd")
add("GateOx_%d" % (i + 1), box(xc, 0, Lg, tox), "ox")
add("Gate_%d" % (i + 1), box(xc, tox, Lg, Hg), "gate")
add("SpacerL_%d" % (i + 1), box(xl, 0, Lsp, zTop_gate), "spacer")
add("SpacerR_%d" % (i + 1), box(xr, 0, Lsp, zTop_gate), "spacer")
pad_and_plug(2 * i + 2, xc + Lg / 2.0, zTop_gate, stag)
stag += padStag

# Body contact (T8) at the bottom
xb = bodyL / 2.0
add("Plug_T8",
Part.makeBox(plugW, W * 0.6, plugH, V(xb - plugW / 2, W * 0.2, -bodyT - plugH)),
"plug")
zpad8 = -bodyT - plugH - padT
add("Terminal_8",
Part.makeBox(padW, W * 0.8, padT, V(xb - padW / 2, W * 0.1, zpad8)), "pad")
pads[8] = (xb, zpad8) # wire leaves from the underside of the pad

# ---------------- Wiring helpers ----------------
def wire_seg(name, p1, p2, ckey):
d = p2 - p1
L = d.Length
if L < 1e-6:
return
add(name, Part.makeCylinder(wr, L, p1, d), ckey)

def joint(name, p, ckey):
add(name, Part.makeSphere(wr * 1.15, p), ckey)

def route(tag, tnum, yw, z_bus, x_end, z_end, ckey):
"""pad -> vertical to z_bus -> horizontal in X to x_end -> vertical to z_end."""
xc, zt = pads[tnum]
p0 = V(xc, yw, zt)
p1 = V(xc, yw, z_bus)
p2 = V(x_end, yw, z_bus)
p3 = V(x_end, yw, z_end)
wire_seg("Wire_%s_a" % tag, p0, p1, ckey)
wire_seg("Wire_%s_b" % tag, p1, p2, ckey)
wire_seg("Wire_%s_c" % tag, p2, p3, ckey)
joint("Joint_%s_1" % tag, p1, ckey)
joint("Joint_%s_2" % tag, p2, ckey)
return p3
```

```
def gnd_symbol(tag, p, direction=-1):
"""Ground symbol (3 plates) hanging from point p, plates pointing down (-1) or up (+1)."""
x, y, z = p.x, p.y, p.z
wire_seg("Wire_%s_gnd" % tag, p, V(x, y, z + direction * 8), "wGND")
for k, s in enumerate((50.0, 34.0, 18.0)):
zc = z + direction * (8 + 3 * k)
add("GND_%s_plate%d" % (tag, k + 1),
Part.makeBox(s, s * 1.2, 3.0, V(x - s / 2, y - s * 0.6, zc - (3.0 if direction < 0 else 0))),
"wGND")
label("GND", x + 35, y, z + direction * 8)

def instrument(tag, x_centre, z_bottom, text):
add("Instrument_%s" % tag,
Part.makeBox(instW, W * 0.9, instH, V(x_centre - instW / 2, W * 0.05, z_bottom)),
"inst")
label(text, x_centre - instW / 2, W * 0.05, z_bottom + instH + 6)

# ---------------- Measurement wiring ----------------
z_max = max(zt for (_, zt) in [pads[t] for t in range(1, 8)])
z_inst = z_max + 120.0

x3 = pads[3][0]
x4 = pads[4][0]
x5 = pads[5][0]

# T3 -> SMU1 (force V_DS, measure I_D), red
x_smu1 = x3 - 70.0
p = route("T3", 3, W * 0.30, z_max + 40.0, x_smu1, z_inst, "wD")
instrument("SMU1", x_smu1, z_inst, "SMU1: force Vds (10-50 mV), measure Id")

# T4 -> SMU2 (gate sweep), blue
x_smu2 = x4
p = route("T4", 4, W * 0.50, z_max + 70.0, x_smu2, z_inst, "wG")
instrument("SMU2", x_smu2, z_inst, "SMU2: sweep Vg (Vg-Vt = 0.3..1.5 V)")

# T5 -> GND, black
x_g5 = x5 + 70.0
p = route("T5", 5, W * 0.70, z_max + 40.0, x_g5, z_max + 40.0, "wGND")
gnd_symbol("T5", p, -1)

# T8 -> GND (common ground with T5), black
z_low = zpad8 - 40.0
x_g8 = xb + 90.0
p = route("T8", 8, W * 0.50, z_low, x_g8, z_low, "wGND")
gnd_symbol("T8", p, -1)

# Floating terminals T1, T2, T6, T7: short open stubs
for t in (1, 2, 6, 7):
xc, zt = pads[t]
p0 = V(xc, W * 0.5, zt)
p1 = V(xc, W * 0.5, zt + stubH)
wire_seg("Stub_T%d" % t, p0, p1, "wF")
add("StubEnd_T%d" % t, Part.makeSphere(wr * 2.2, p1), "wF")
label("T%d float" % t, xc - 12, W * 0.5, zt + stubH + 8)

# ---------------- Terminal labels + title ----------------
for t in range(1, 8):
xc, zt = pads[t]
label("T%d" % t, xc - 5, -10.0, zt + 2)
label("T8", pads[8][0] - 5, -10.0, zpad8 - 12)
label(["LDD extraction on M2 (T3-T5):", "R_tot = 2*R_LDD + R_ch(Vg)",
"plot R_tot vs 1/(Vg-Vt); intercept = 2*R_LDD"],
0, -10.0, z_inst + instH + 40)

doc.recompute()

if App.GuiUp:
import FreeCADGui as Gui
Gui.ActiveDocument.ActiveView.viewFront()
Gui.SendMsgToActiveView("ViewFit")

App.Console.PrintMessage(
"Done. Red=T3 (SMU1, force Vds/measure Id), Blue=T4 (SMU2, Vg sweep), "
"Black=T5 & T8 to GND, Gray=T1/T2/T6/T7 floating.\n")
```

Listing 3. Python FreeCAD macro for constructing the three-MOSFET LDD structure and visualizing the proposed measurement connections.

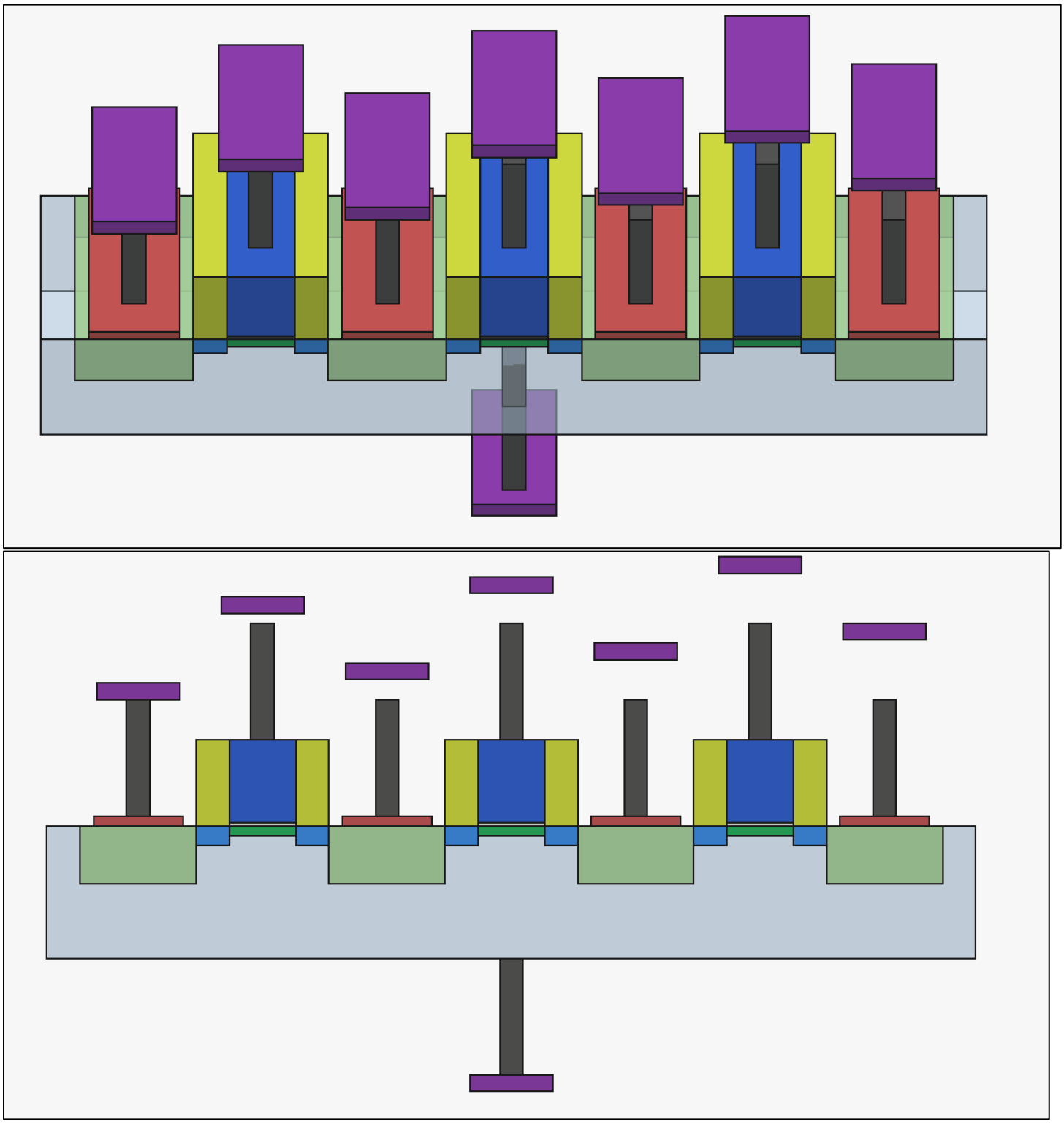


Figure 7. Three-dimensional FreeCAD model of three adjacent MOSFET structures for LDD-resistance extraction, showing the source/drain regions, NiSi contacts, LDD extensions, channel and gate regions, terminal pads, and body contact.

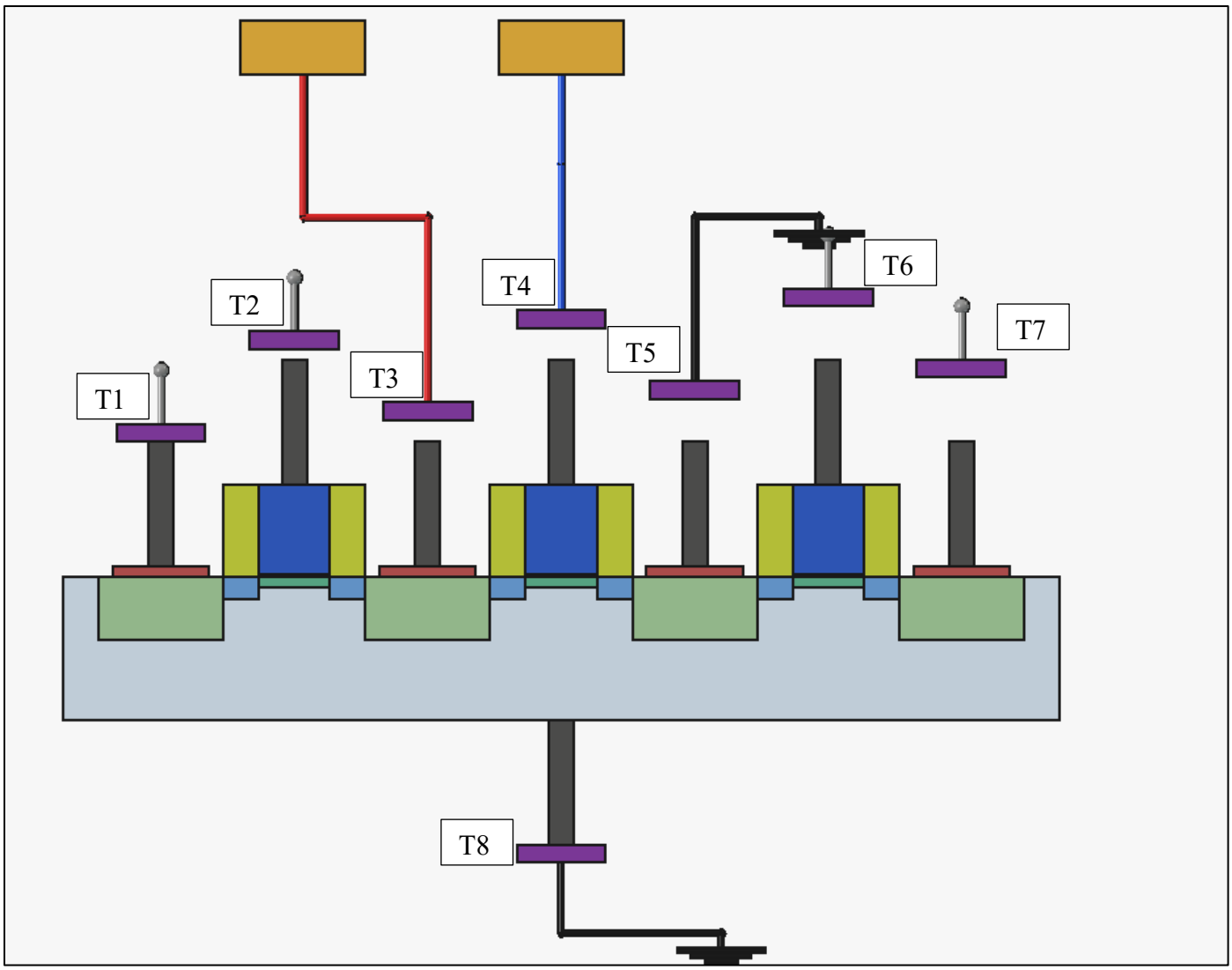


Figure 8. Proposed measurement configuration for extracting the LDD resistance of the middle MOSFET (M2). Terminal T3 connects to SMU1 for drain–source biasing and current measurement, T4 connects to SMU2 for gate-voltage sweeping, and T5 and T8 are grounded. Terminals T1, T2, T6, and T7 are shown as floating connections.

The measurement procedure:

- **T3 (red wire):** goes to SMU1, which forces a small V_{DS} (10–50 mV) and measures I_D.
- **T4 (blue wire):** goes to SMU2, which sweeps the gate of the middle device ($V_G - V_T$ from 0.3 to 1.5 V).
- **T5 and T8 (black wires):** both go to a ground symbol, so the S/D of M2 and the body share a common ground.
- **T1, T2, T6, T7 (gray stubs):** each ends in an open sphere labeled “float”, so M1 and M3 are not in the measurement path.

Appendix:

Commands and Tools Used:

Tool	Used for	Commands / calls
FreeCAD (Part, Draft, Spreadsheet workbenches)	Problem 1 pad and device layout; Problem 2 3-D device and wiring schematic	*Macro > Macros… > Execute (or paste into the Python console); App.newDocument(), Part.makeBox / makeLine / makeCylinder / makeSphere, Draft.make_text, doc.addObject("Spreadsheet::Sheet", …), doc.recompute(), Gui.activeDocument().activeView().viewTop() / viewFront(), Gui.SendMsgToActiveView("ViewFit")*
Python 3, NumPy, Matplotlib (Windows PowerShell)	Problem 2 R_tot vs 1/(V_G − V_T) fit and plot	*py -m pip install numpy matplotlib, py ldd_resistance_extraction.py, np.polyfit(x, Rtot, 1), fig.savefig("ldd_resistance_extraction_example.png", dpi=200)*
Source-measure units (SMUs)	Measurement methodology in both problems	Conceptual only: *force/sense (Kelvin) connections, remote sense, V_DS = 10–50 mV, V_G sweep*
AI assistant (Claude, Anthropic)	Drafting the pad-allocation reasoning and the FreeCAD macros	The macro logic was also checked against a stubbed FreeCAD API.

Table 4. Summary of software tools, libraries, measurement concepts, and principal commands used to construct the FreeCAD MOSFET layouts, generate the LDD-resistance extraction plot, and describe the proposed source-measure-unit measurement procedures.

Executive Summary:

This report addresses two problems from the Micron Open Day hackathon.

Problem 1: maximum MOSFETs on a 14-pad module.

- Each device is measured with Kelvin (force and sense) drain and source connections to cut the effect of interconnect resistance. Gate and bulk carry essentially no DC current, so they are shared and need no sense line.
- Four shared pads (G, B, S-force, S-sense) plus two pads per device (D-force, D-sense) give $4 + 2n \leq 14$, so $n = 5$ MOSFETs using all 14 pads.
- Only one device is measured at a time, and the other drains float.
- A FreeCAD macro builds the layout with each MOSFET in the 70 µm gap between its own D-force and D-sense pads. The drain traces are about 35 µm long and join at the drain contact.
- The macro adds a dimensioned top view (footprint 1890 × 158 µm), an 8:1 detail of one gap, and a design-rule margin table. The model is conceptual, and its rule values are placeholders.

Problem 2: LDD resistance from three adjacent MOSFETs.

- The middle device is measured in the linear regime ($V_{DS} \approx 10$–50 mV) with the body grounded.
- The total resistance is $R_{tot}(V_G) = R_{NiSi} + 2R_{SD} + 2R_{LDD} + R_{ch}(V_G)$, with $R_{ch} \propto 1/(V_G - V_T)$.
- Plotting R_{tot} against $1/(V_G - V_T)$ and extrapolating to zero gives R_{ext}, so $R_{LDD} = R_{ext} / 2$ (more exactly, $(R_{ext} - R_{NiSi} - 2R_{SD})/2$).
- In the worked example, an intercept of 199.83 Ω gives $R_{LDD} \approx 74.9$ Ω after subtracting $R_{NiSi} = 10$ Ω and $R_{SD} = 20$ Ω per side.
- Checks on the outer devices, reverse bias, and body bias test the assumptions.

Limitations:

Problem 1: design

- All devices must be the same type (for example NMOS) to share one body. Mixed NMOS and PMOS would need separate bulk pads.
- Kelvin sensing removes interconnect IR drop from the voltage measurement, but not from the force path. In the macro's placeholder estimate, the longest shared source path (to M5) is about 25 Ω, or about 126 mV at 5 mA. The SMU compliance must cover this, and the bus may need widening.
- The shared gate carries the leakage of all five devices, so the source current equals I_D only if gate leakage is negligible.
- Unselected devices are assumed inactive because their drains float. Junction leakage is neglected.
- Sense accuracy depends on high-impedance sense inputs and low leakage on those lines.
- The source Kelvin point is a shared diffusion contact. It is not a fully separate force/sense tap.
- The 4 + 2n bound assumes every device needs its own D-force and D-sense pads and that no on-chip switching is used.

Problem 1: FreeCAD macro

- The model is a conceptual visualization, not a foundry-rule-compliant physical layout.
- The rule values are generic placeholders, and a pass does not mean DRC/LVS clean.
- It has no wells, implants, STI, guard rings, passivation opening, GDS layers, or manufacturing grid. It uses single oversized contacts and vias instead of arrays.
- Current-density and resistance numbers use assumed values (J_M1 = 1 mA/μm, J_M2 = 2 mA/μm, Rs of 0.09 and 0.08 Ω/sq, 5 mA test current).
- The macro creates about 300 text objects, so building can be slow.
- The macro logic was checked against a stubbed FreeCAD API. The real run is shown in the report screenshots, and the text styling may still need tuning.

Problem 2: method

- $R_{NiSi} \ll R_{LDD}$ is given. R_SD (deep S/D) is not separated by this structure alone. If it is not negligible, as with the 20 Ω per side in the worked example, it needs an independent estimate.
- The linear extrapolation assumes the LDD resistance saturates at high gate overdrive and is bias-independent. Mobility degradation, series-resistance coupling, and the L_eff versus L_g difference can bend the line, so the fit must stay in the high-overdrive range.
- V_T must be extracted first, and any V_T error shifts the $1/(V_G - V_T)$ axis.
- A two-terminal $R_{tot} = V_{DS} / I_D$ also includes probe and contact resistance. Symmetry between the source-side and drain-side LDD is assumed.
- The plot uses synthetic data, not measurements.

Problem 2: FreeCAD schematic

- Device dimensions and wiring are schematic and not to scale.
- The macro is for explanation only and has no electrical simulation.

Conclusion:

A 14-pad module supports at most five 4-terminal MOSFETs with Kelvin drain and source connections, because four pads are shared and each device needs two. Placing each device in the 70 μm gap between its own drain pads keeps the drain interconnect short, and the dimensioned FreeCAD model with its margin table makes the topology easy to review. The model is conceptual, so a real tape-out still needs a PDK-compliant layout.
For Problem 2, one gate-voltage sweep on the middle device gives R_ext, the gate-independent resistance, from the R_{tot} versus $1/(V_G - V_T)$ intercept. With $R_{NiSi} \ll R_{LDD}$, $R_{LDD} \approx R_{ext} / 2$. The cross-checks on M1 and M3, bias polarity, and body bias test the assumptions behind this.

Future Scope:

- Redraw the Problem 1 layout in KLayout or Virtuoso with a real PDK. This adds wells, guard rings, exact-size contact and via arrays, a pad cell with passivation opening and ESD, and DRC/LVS/parasitic extraction with electromigration and IR sign-off.
- Increase the device count beyond 5 by adding on-chip select switches or a decoder. This needs analysis so the switches do not add error to the force and sense paths.
- Widen the shared source bus, or add a second source-force path, to reduce force-path IR drop at higher currents.
- Use the outer devices as zero-current sense paths for T3 and T5 (see the first edit below). This would remove the probe and NiSi contact drop from V_DS.
- Separate R_SD from R_LDD with a second structure or a channel-length series (varying L_g) in the same fit.
- Replace the synthetic Problem 2 data with measured data, and add a mobility-degradation or Y-function fit with uncertainty bars.
- Export the macro geometry to CSV and then to GDS (for example with gdstk) as a bridge to the layout tools.